7 性能分析工具概述

在上一章中，我们探讨了现代处理器中实现的用于辅助性能分析的功能。然而，如果你直接开始使用这些功能，很快就会变得非常微妙，因为需要大量的低级编程才能利用它们。幸运的是，性能分析工具处理了所有必要的复杂性，以有效地使用这些硬件性能监控功能。它使分析变得顺利，但了解工具如何获取和解释数据是至关重要的。这就是为什么我们在讨论 CPU 性能监控功能之后再讨论分析工具的原因。

本章简要介绍了主要平台上最流行的工具。选择将取决于你使用的操作系统和 CPU。一些工具是跨平台的，但大多数不是，因此了解哪些工具对你可用至关重要。这些分析工具通常由硬件供应商自己开发和维护，因为他们是唯一了解如何正确使用其 CPU 上可用性能监控功能的人。不幸的是，这造成了这样一种情况：如果你需要进行高级性能工程工作，你需要安装一个依赖于你使用的 CPU 的专门工具。

阅读完本章后，花时间练习使用你可能最终会使用的工具。熟悉这些工具的界面和工作流程。对你日常工作中使用的应用程序进行性能分析。即使你找不到任何可操作的见解，当真正需要时，你会更加做好准备。

7.1 Intel Vtune

VTune Profiler（之前称为 VTune Amplifier）是一款适用于基于 x86 架构的机器的性能分析工具，具有丰富的图形用户界面。它可以在 Linux 或 Windows 操作系统上运行。我们跳过了关于 VTune 对 MacOS 的支持的讨论，因为它不适用于苹果芯片（例如，M1 和 M2），而且基于英特尔的 MacBook 很快就会过时。

Vtune 可以在英特尔和 AMD 系统上使用，许多功能都可以工作。但是，高级基于硬件的采样需要英特尔制造的 CPU。例如，在 AMD 系统上，你将无法使用 Intel Vtune 收集硬件性能计数器。

截至 2023 年初，VTune 作为一个独立工具或作为英特尔 oneAPI 基础工具包的一部分免费提供。

如何配置

在 Linux 上，Vtune 可以使用两个数据收集器：Linux perf 和 Vtune 自己的驱动程序，称为 SEP。第一种类型用于用户模式采样，但如果你想进行高级分析，你需要构建并安装 SEP 驱动程序，这并不太难。

```
# 进入 vtune 安装的 sepdk 文件夹
$ cd ~/intel/oneapi/vtune/latest/sepdk/src
# 构建驱动程序
$ ./build-driver
# 添加 vtune 组并将你的用户添加到该组
# 创建一个新的 shell，或者重新启动系统
$ sudo groupadd vtune
$ sudo usermod -a -G vtune `whoami`
# 安装 sep 驱动程序
$ sudo ./insmod-sep -r -g vtune
```

完成上述步骤后，你应该能够使用像微架构探索和内存访问之类的高级分析类型。

在 Windows 上，在安装 Vtune 后不需要进行任何额外的配置。收集硬件性能事件需要管理员权限。

它能做什么

- 找到热点：函数、循环、语句。

- 监控各种特定于 CPU 的性能事件，例如分支误判和 L3 缓存未命中。
- 定位这些事件发生的代码行。
- 使用 TMA 方法论对 CPU 性能瓶颈进行特征化。
- 为特定函数、进程、时间段或逻辑核心过滤数据。
- 随时间观察工作负载行为（包括 CPU 频率、内存带宽利用率等）。

VTune 可以提供关于运行中进程的非常丰富的信息。如果你想要提高应用程序的整体性能，这是一个合适的工具。Vtune 总是提供一段时间内的聚合数据，因此它可用于找到“平均情况”下的优化机会。

它不能做什么

- 分析非常短的执行异常。
- 观察系统范围的复杂软件动态。

由于该工具的采样性质，它最终会错过持续时间非常短的事件（例如，亚微秒级）。

示例

以下是 VTune 最有趣的功能的一系列截图。为了这个示例，我们使用了 POV-Ray，一个用于创建 3D 图形的光线追踪器。图 43 显示了 povray 3.7 的内置基准测试的热点分析，该基准测试使用 clang14 编译器编译，并使用 `-O3 -ffast-math -march=native -g` 选项在英特尔 Alderlake 系统上运行（Core i7-1260P，4 个 P 核 + 8 个 E 核），并使用 4 个工作线程。

在图像的左侧部分，你可以看到工作负载中一系列热点函数，以及相应的 CPU 时间百分比和退役 (retired) 指令的数量。在右侧面板中，你可以看到导致调用函数 `pov::Noise` 的最频繁的调用栈之一。根据该截图，`44.4%` 的时间函数 `pov::Noise` 是从 `pov::Evaluate_TPat` 被调用的，而 `pov::Evaluate_TPat` 又是从 `pov::Compute_Pigment` 被调用的。请注意，调用栈并没有一直指向 `main` 函数。这是因为使用基于硬件的收集时，VTune 使用 LBR 来采样调用栈，其深度有限。在这里很可能涉及递归函数，要进一步调查，用户必须深入代码。

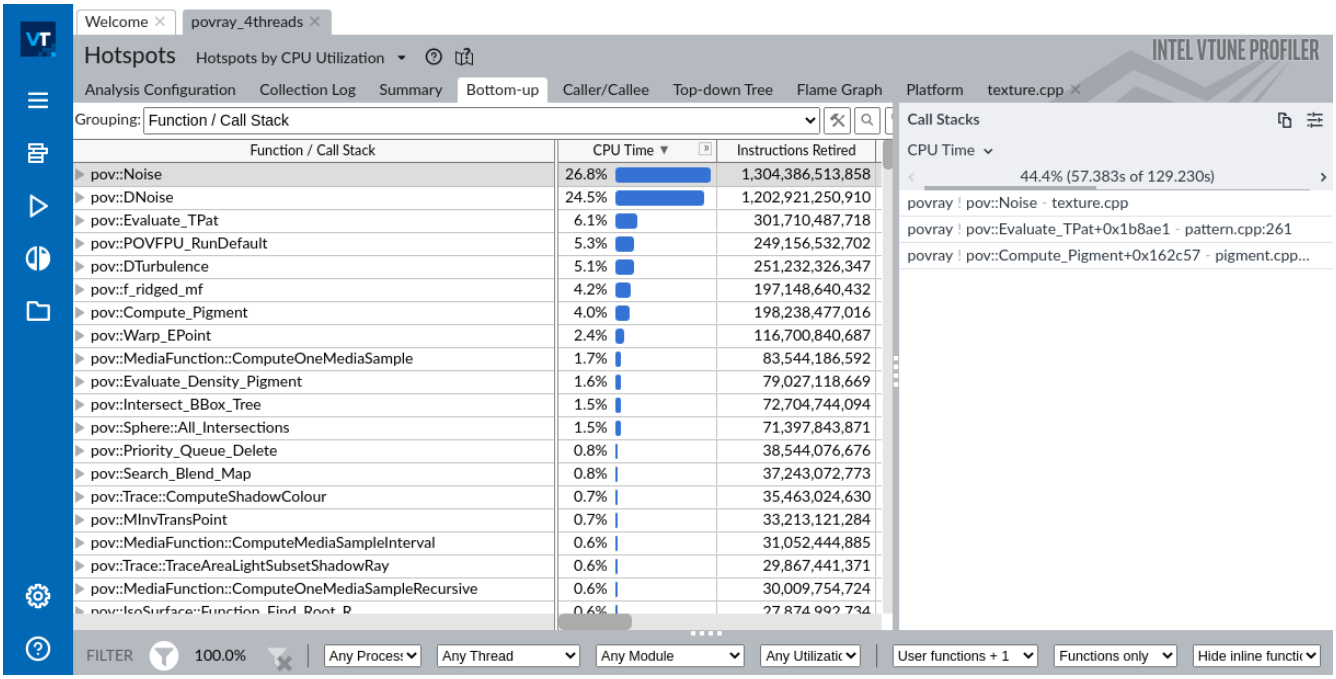


Figure 43: VTune 对 povray 内置基准测试的热点视图。

如果你双击 `pov::Noise` 函数，你会看到图 44 所示的图像。出于篇幅考虑，只显示了最重要的列。左侧面板显示了

源代码和每行代码对应的 CPU 时间。右侧，你可以看到一些汇编指令以及它们被归因的 CPU 时间。在左侧面板中高亮显示的机器指令对应于右侧面板中的第 476 行。两个面板中所有 CPU 时间百分比的总和等于归因给 `pov::Noise` 的总 CPU 时间，即 26.8%。

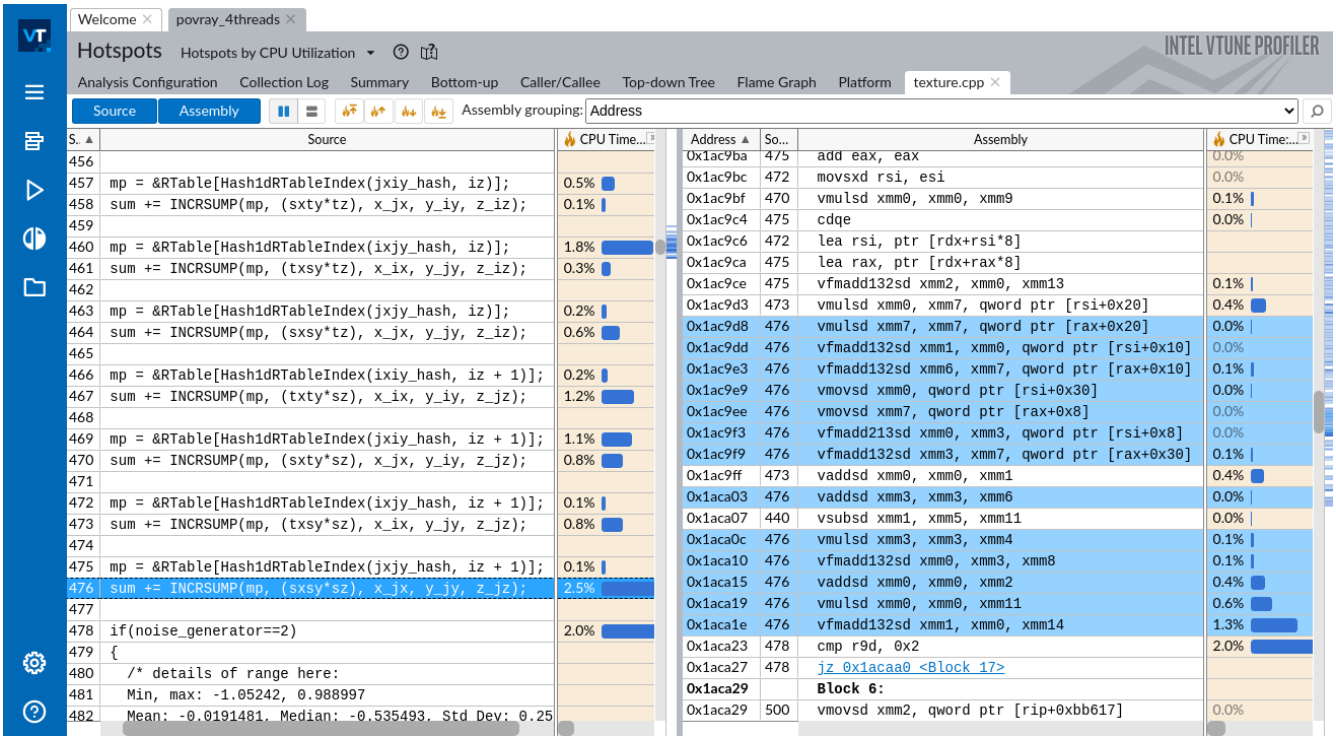


Figure 44: VTune 对 povray 内置基准测试的源代码视图。

当你使用 VTune 来分析运行在英特尔 CPU 上的应用程序时，它可以收集许多不同的性能事件。为了说明这一点，我们运行了不同的分析类型，微架构探索，我们已经在上一章中展示过。那时我们用它来进行自顶向下的微架构分析，而我们也可以用它来观察原始性能事件。要访问原始事件计数，可以将视图切换为硬件事件，如图 45 所示。要启用视图切换，你需要在选项 -> 通用 -> 显示所有适用的视角 中选中标记。还有两个有用的页面，没有显示在图像上：摘要页面提供了从 CPU 计数器收集的原始性能事件的绝对数目，事件计数页面提供了相同数据的函数级别细分。读者可以自行尝试并查看这些视图。

图 45 相当繁忙，需要一些解释。顶部面板（区域 1）是一个时间轴视图，显示了我们四个工作线程随时间的行为，关于 L1 缓存未命中，以及主线程（TID：3102135）的一些微小活动，它生成所有工作线程。黑色条的高度越高，每时每刻发生的事件（在这种情况下是 L1 缓存未命中）就越多。注意到所有四个工作线程的 L1 未命中偶发性峰值。我们可以使用这个视图来观察工作负载的不同或可重复的阶段。然后为了弄清楚在那个时间执行了哪些函数，我们可以选择一个时间间隔，并点击“筛选进来”以只关注运行时间的那一部分。区域 2 就是这种过滤的一个示例。要查看更新后的函数列表，你可以转到自下而上或，在这种情况下，事件计数视图。这样的过滤和缩放功能在所有 Vtune 时间轴视图中都可用。

区域 3 显示了收集的性能事件及其随时间的分布。这次不是 perf-thread 视图，而是在所有线程上聚合的数据。除了观察执行阶段外，你还可以从中直观地提取一些有趣的信息。例如，我们可以看到执行的分支数很高（`BR_INST_RETIRED.ALL_BRANCHES`），但误判率相当低（`BR_MISP_RETIRED.ALL_BRANCHES`）。这可能导致你得出结论，分支误判不是 POV-Ray 的瓶颈。如果你向下滚动，你会看到 L3 未命中的数量为零，而 L2 缓存未命中也非常罕见。这告诉我们，在 99% 的时间里，内存访问请求由 L1 处理，并且其余部分由 L2 处理。两个观察结合起来，我们可以得出结论，应用程序很可能受到计算的限制，即 CPU 忙于计算某些东西，而不是在等待内存或从误判中恢复。

最后，底部面板（区域 4）显示了四个硬件线程的 CPU 频率图表。悬停在不同的时间片上告诉我们，这些核心的频

率在 3.2GHz - 3.4GHz 区域波动。内存访问分析类型还显示了随时间变化的内存带宽（以 GB/s 为单位）。

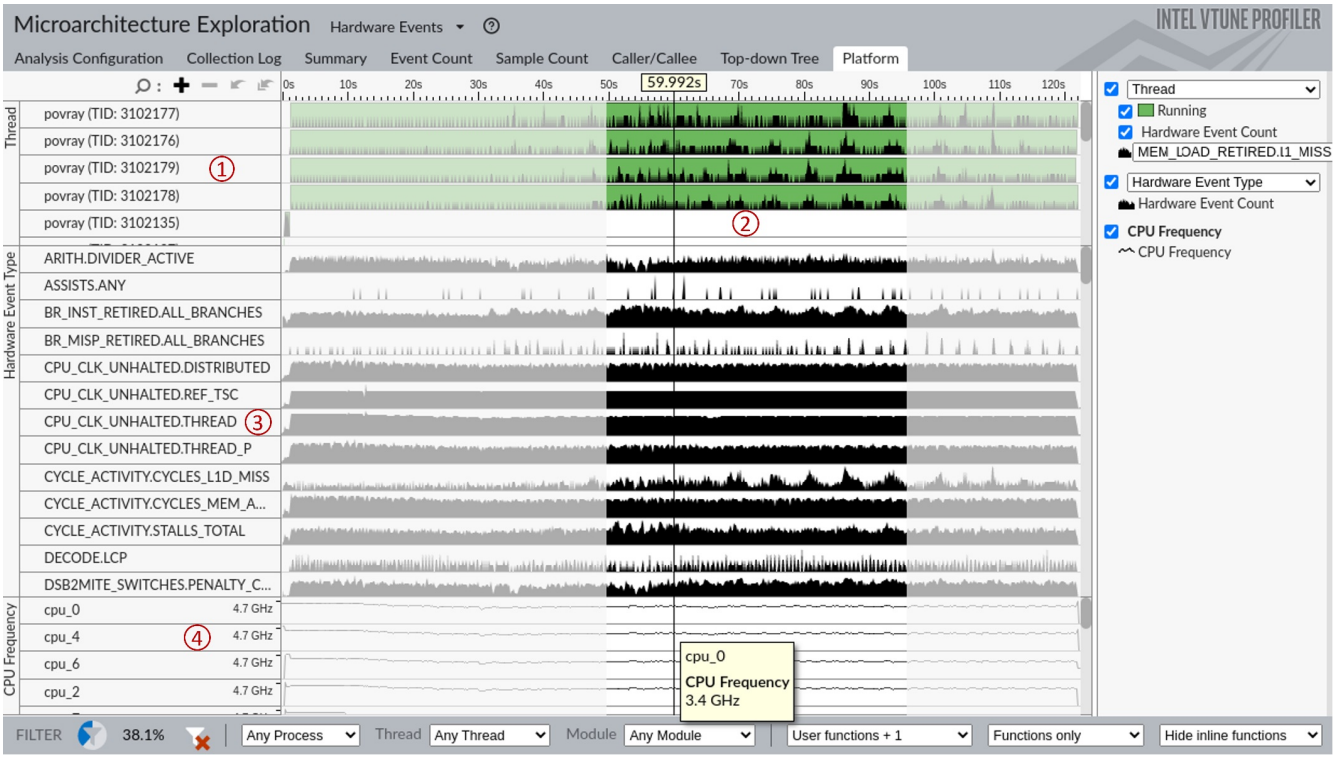


Figure 45: VTune 对 povray 内置基准测试的性能事件时间轴视图。

Intel® VTune™ Profiler 中的 TMA

TMA 通过最新的 Intel VTune Profiler 中的“微架构探索”[110] 分析进行展示。图 46 显示了 7-zip 基准测试[111] 的分析摘要。在图表中，你可以看到由于 CPU `Bad Speculation`（错误推测）以及特别是由于误判的分支，大量的执行时间被浪费了。

该工具的美妙之处在于，你可以点击你感兴趣的指标，工具会带你到显示对该特定指标做出贡献的顶级函数的页面。例如，如果你点击 `Bad Speculation` 指标，你会看到类似于图 47 所示的内容。[112]

从那里，如果你双击 `LzmaDec_DecodeReal2` 函数，Intel® VTune™ Profiler 将带你到类似图 48 所示的源码级别视图。高亮显示的行在 `LzmaDec_DecodeReal2` 函数中贡献了最大数量的分支误判。

7.2 AMD uProf

AMD uProf 是由 AMD 开发的一款用于监视在 AMD 处理器上运行的应用程序性能的工具。虽然 uProf 也可以用于 Intel 处理器，但你只能使用 CPU 无关的功能。该分析器可以免费下载，并可在 Windows、Linux 和 FreeBSD 上使用。AMD uProf 可用于在多个虚拟机（VM）上进行分析，包括 Microsoft Hyper-V、KVM、VMware ESXi、Citrix Xen，但并非所有 VM 上的所有功能都可用。此外，uProf 还支持分析使用各种语言编写的应用程序，包括 C、C++、Java、.NET/CLR。

[110] VTune 微架构分析 - https://software.intel.com/en-us/vtune-help-general-exploration-analysis。在 Intel® VTune Profiler 的 2019 年之前的版本中，它被称为“通用探索”分析。

[111] 7zip 基准测试 - https://github.com/llvm-mirror/test-suite/tree/master/MultiSource/Benchmarks/7zip。

[112] TMA 指标的每个函数视图是 Intel®VTune profiler 独有的功能。

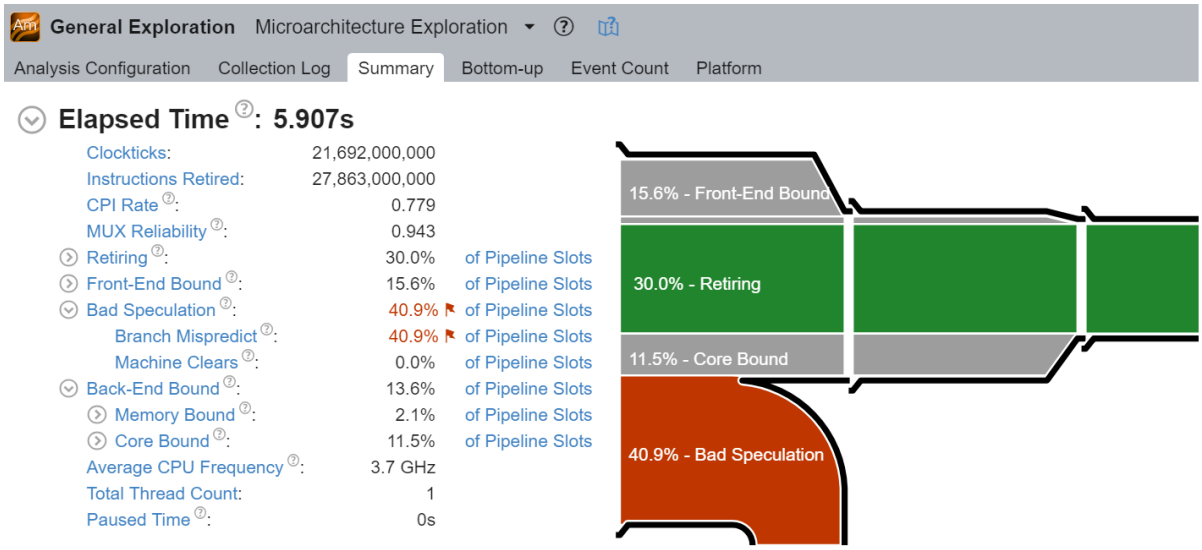


Figure 46: Intel VTune Profiler 中的“微架构探索”分析。

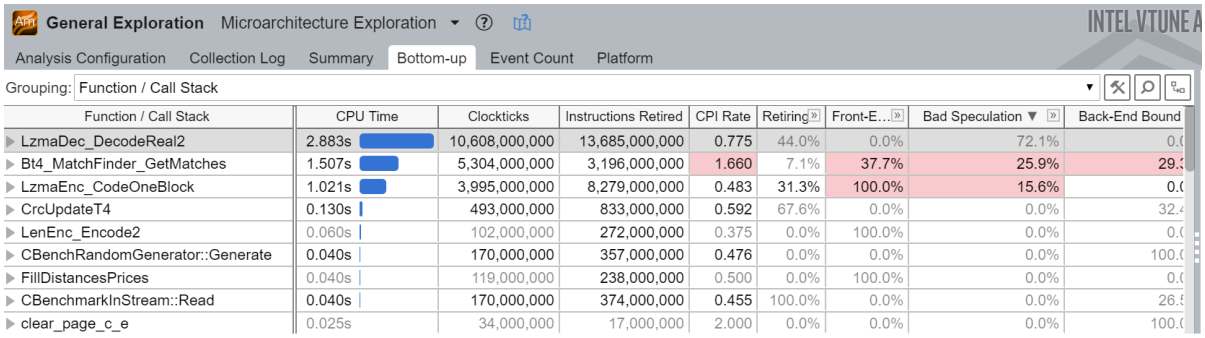


Figure 47: “微架构探索”自下而上视图。

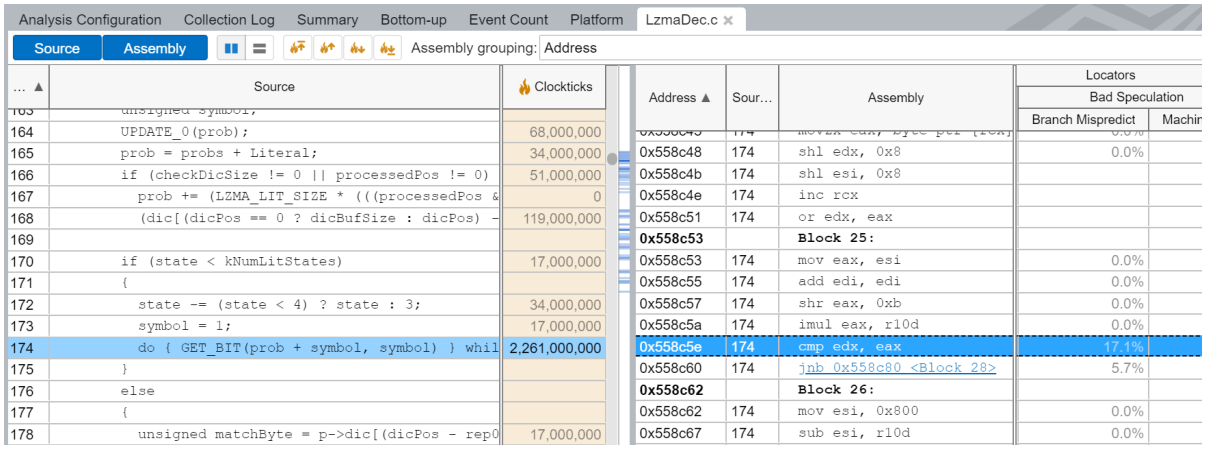


Figure 48: “微架构探索”源代码和汇编视图。

如何配置

在 Linux 上，uProf 使用 Linux perf 进行数据收集。在 Windows 上，uProf 使用自己的采样驱动程序，在安装 uProf 时会安装该驱动程序，无需额外配置。AMD uProf 支持命令行界面（CLI）和图形界面（GUI）。CLI 界面需要两个单独的步骤 - 收集和报告，类似于 Linux perf。

可以做什么

- 找到热点：函数、语句、指令。
- 监视各种硬件性能事件并定位发生这些事件的代码行。
- 为特定函数或线程过滤数据。
- 观察工作负载的行为随时间的变化：在时间轴图中查看各种性能事件。
- 分析热门调用路径：调用图、火焰图和自下而上图表。

此外，uProf 还可以监视 Linux 上的各种 OS 事件 - 线程状态、线程同步、系统调用、页面错误等。它还允许分析 OpenMP 应用程序以检测线程不平衡，并分析 MPI 应用程序以检测 MPI 群集节点之间的负载不平衡。关于 uProf 各种功能的更多详细信息可在用户指南[113]中找到。

不能做什么

由于工具的采样性质，它最终会错过持续时间非常短的事件。报告的样本是统计估算的数字，大多数情况下足以分析性能，但不是事件的精确计数。

示例

为了展示 AMD uProf 工具的外观和感觉，我们在 AMD Ryzen 9 7950X、Windows 11、64 GB RAM 上运行了 Scimark2[114] 基准测试中的密集 LU 矩阵分解组件。

图 49 显示了函数热点分析。在图像的顶部，你可以看到事件时间轴，显示了在应用程序执行的不同时间观察到的事件数量。在右侧，你可以选择要绘制的指标，我们选择了 `RETIRED_BR_INST_MISP`。注意在时间范围从 20s 到 40s 的分支误判的峰值。你可以选择该区域以密切分析那里发生了什么。一旦你这样做了，它将更新底部面板，仅显示该时间间隔的统计信息。

在时间轴图下方，你可以看到一系列热点函数，以及相应的采样性能事件和计算的指标。事件计数可以显示为：样本计数、原始事件计数和百分比。有许多有趣的数字可以查看，然而，我们不会深入分析，但鼓励读者找出分支误判的性能影响并找到其源头。

在函数表下面，你可以看到所选函数在函数表中的自底向上调用栈视图。正如我们所见，所选的 `LU_factor` 函数是从 `kernel_measureLU` 被调用的，而 `kernel_measureLU` 又是从 `main` 被调用的。在 Scimark2 基准测试中，这是 `LU_factor` 的唯一调用栈，即使它显示了 `Call Stacks [5]`，这是可以忽略的采集工件。但在其他应用程序中，热点函数可以从许多不同的位置调用，因此你可能还想检查其他调用栈。

如果你双击任何函数，uProf 将打开该函数的源代码/汇编视图。为了简洁起见，我们不显示此视图。在左侧面板上，还有其他视图可用，如指标、火焰图、调用图

视图和线程并发。它们也对分析很有用，但我们决定跳过它们。读者可以自行尝试并查看这些视图。

[113] AMD uProf 用户指南 - https://www.amd.com/en/developer/uprof.html#documentation

[114] Scimark2 - https://math.nist.gov/scimark2/index.html

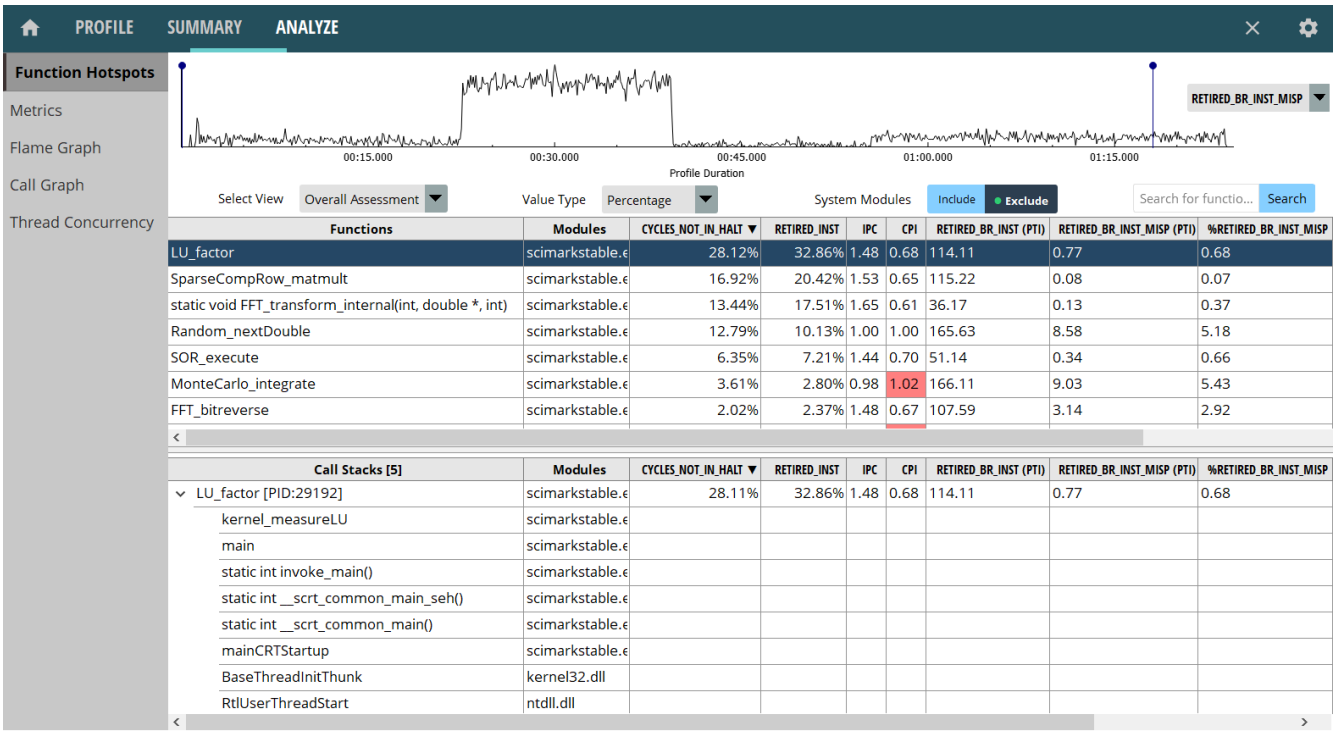


Figure 49: uProf 的函数热点视图。

7.3 Apple Xcode Instruments

在 MacOS 上进行初始性能分析最方便的方法是使用 Instruments。它是一个应用程序性能分析器和可视化工具，与 Xcode 一起免费提供。Instruments 建立在从 Solaris 移植到 MacOS 的 DTrace 追踪框架之上。Instruments 拥有许多工具，可用于检查应用程序的性能，并允许我们执行大多数其他性能分析器（如 Intel Vtune）可以执行的基本操作。获取分析器的最简单方法是从 Apple AppStore 安装 Xcode。该工具无需配置，安装后即可立即使用。

在 Instruments 中，你可以使用专门的工具（称为 instruments）来跟踪应用程序、进程和设备的不同方面的情况随时间的变化。Instruments 拥有强大的可视化机制。它在进行分析时收集数据，并实时向你展示结果。你可以收集不同类型的数据，并将它们并排显示，这使你可以看到执行中的模式，相关系统事件，并发现非常微妙的性能问题。

在本章中，我们只展示了“CPU 计数器”工具，这是本书最相关的工具之一。Instruments 还可以可视化 GPU、网络和磁盘活动，跟踪内存分配和释放，捕获用户事件（如鼠标点击），提供关于功耗效率的见解等。关于这些用例的更多信息可以在 Instruments 的文档[115]中找到。

你可以用它做什么

- 访问 Apple M1 和 M2 处理器上的硬件性能计数器
- 找到程序中的热点以及调用栈
- 与源代码并排检查生成的 ARM 汇编代码
- 为时间轴上的选定区间筛选数据

[115] Instruments 文档 - https://help.apple.com/instruments/mac/current

你不能用它做什么

示例：编译 Clang

正如我们所宣传的那样，在此示例中，我们将展示如何在搭载 M1 处理器的 Apple Mac mini 上，macOS 13.5.1 Ventura，16 GB RAM 上收集 HW 性能计数器。我们选取了 LLVM 代码库中最大的文件之一，并使用 Clang C++ 编译器版本 15.0 对其进行编译。以下是我们将要进行分析的命令行：

```
$ clang++ -O3 -DNDEBUG -arch arm64 <other options ...> -c
    llvm/lib/Transforms/Vectorize/LoopVectorize.cpp
```

首先，打开 Instruments 并选择 “CPU 计数器” 分析类型。（这里我们需要稍微提前一点）。它将打开主时间轴视图，如图 51 所示，准备开始分析。但在开始之前，让我们配置收集。单击并按住红色目标图标 ①，然后选择 “Recording Options…” 菜单。它将显示如图 50 所示的对话框窗口。在这里，你可以为收集添加 HW 性能监视事件。

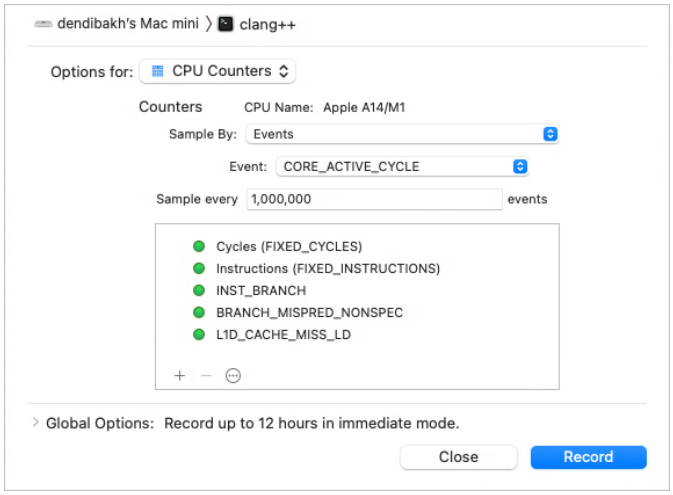


Figure 50: Xcode Instruments：CPU 计数器选项。

据我们所知，Apple 没有在线记录他们的 HW 性能监视事件，但他们在 `/usr/share/kpep` 中提供了一些带有最小描述的事件列表。有 `plist` 文件，你可以将其转换为 json。例如，对于 M1 处理器，可以运行：

```
$ plutil -convert json /usr/share/kpep/a14.plist -o a14.json
```

然后使用简单的文本编辑器打开 `a14.json`。

第二步是设置分析目标。要做到这一点，单击并按住应用程序的名称（在图 51 中标记为 ②），然后选择你感兴趣的应用程序，如果需要，设置参数和环境变量。现在，你已经准备好开始收集了，请按红色目标图标 ①。

Instruments 显示一个时间轴，并不断更新有关正在运行的应用程序的统计信息。一旦程序完成，Instruments 将显示类似于图 51 所示的图像。编译花费了 7.3 秒，我们可以看到事件量随时间的变化。例如，分支错误预测在运行时末尾变得更加明显。你可以放大时间轴上的该区间，以检查所涉及的函数。

底部面板显示了数值统计信息。要检查类似于 Intel Vtune 的自下而上视图的热点，选择菜单 ③ 中的 “Profile”，然后单击菜单 ④ 中的 “Call Tree” 并选中 “Invert Call Tree” 复选框。这正是我们在图 51 中所做的。

Instruments 显示原始计数以及相对于总数的百分比，如果你想计算次要指标（如 IPC、MPKI 等），则非常有用。在右侧，我们有函数 `llvm::FoldingSetBase::FindNodeOrInsertPos` 的热调用栈。如果你双击一个函数，则可以查看为源代码生成的 ARM 汇编指令。

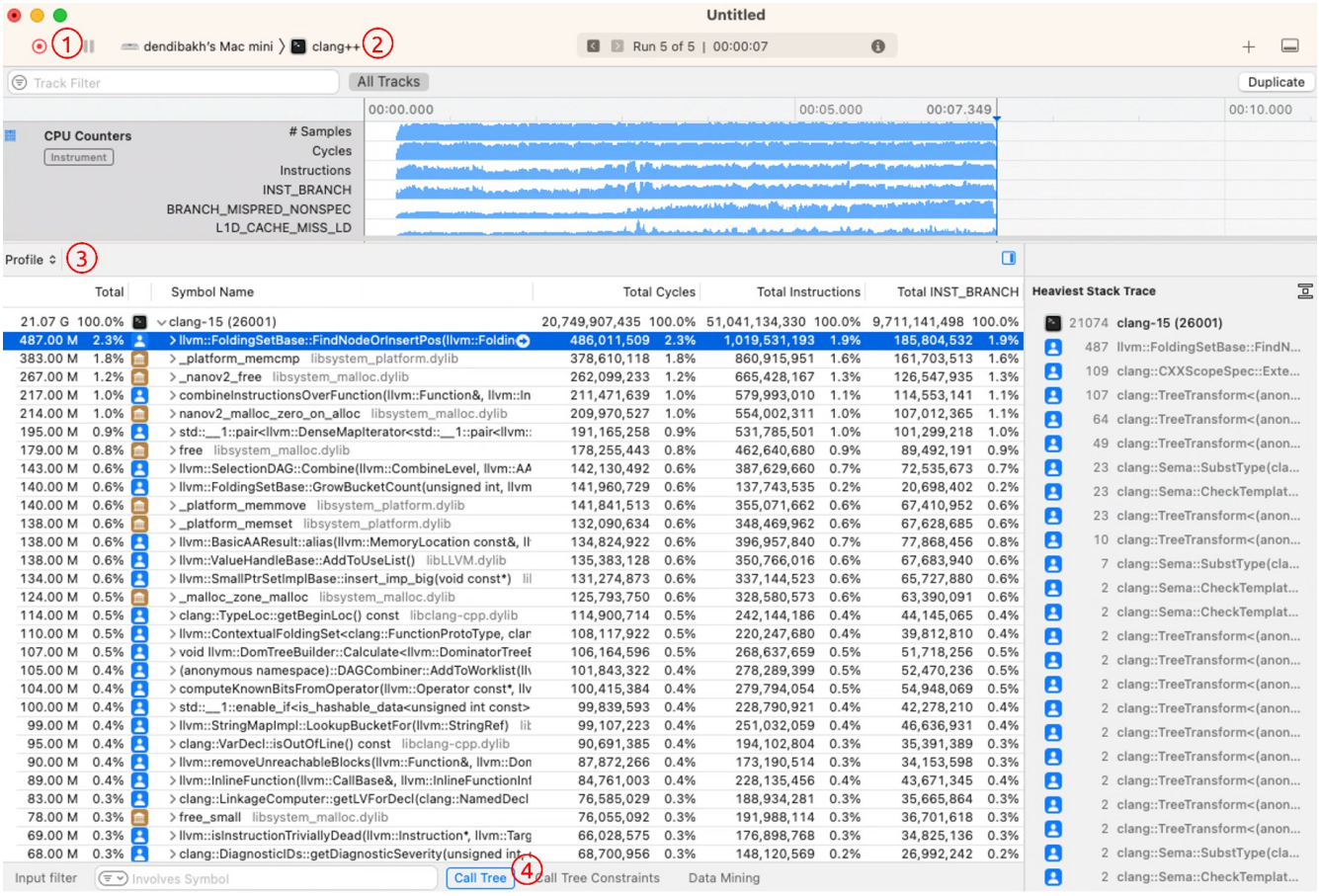


Figure 51: Xcode Instruments：时间轴和统计面板。

据我们所知，MacOS 平台上没有类似质量的替代性分析工具。高级用户可以通过编写简短（或长）的命令行脚本来使用 dtrace 框架本身，但这超出了本书的范围。

7.4 Linux Perf

Linux Perf 是世界上使用最广泛的性能分析器之一，因为它可以在大多数 Linux 发行版上使用，这使其适用于广泛的用户。Perf 在许多流行的 Linux 发行版中都原生支持，包括 Ubuntu、Red Hat、Debian 等。它包含在内核中，因此您可以在任何运行 Linux 的系统上获取操作系统级别的统计信息（页面错误、cpu 迁移等）。截至 2023 年中期，该分析器支持 x86、ARM、PowerPC64、UltraSPARC 和其他一些架构。[116] 这允许访问硬件性能监控功能，例如性能计数器。有关 Linux perf 的更多信息，请访问其维基页面 [117]。

如何配置它

安装 Linux perf 非常简单，只需一个命令即可完成：

```
$ sudo apt-get install linux-tools-common linux-tools-generic linux-tools-`uname -r`
```

另外，除非安全是您关注的问题，否则请考虑更改以下默认设置：

```
# 允许非特权用户进行内核分析和访问 CPU 事件
$ echo 0 | sudo tee /proc/sys/kernel/perf_event_paranoid
$ sudo sh -c 'echo kernel.perf_event_paranoid=0 >> /etc/sysctl.d/local.conf'
# 启用内核模块符号解析以供非特权用户使用
$ echo 0 | sudo tee /proc/sys/kernel/kptr_restrict
$ sudo sh -c 'echo kernel.kptr_restrict=0 >> /etc/sysctl.d/local.conf'
```

它能做什么？

通常，Linux perf 可以完成其他分析器所能做的大多数事情。硬件供应商优先在 Linux perf 中启用他们的功能。因此，当新的 CPU 上市时，perf 已经支持它了。大多数用户将使用两个主要命令：perf stat 和 perf record + perf report。第一个以计数模式收集性能事件的绝对数量，第二个以采样模式分析应用程序或系统。

perf record 命令的输出是原始的示例转储。许多工具构建在 Linux perf 之上，用于解析转储文件并提供新的分析类型。以下是其中最值得注意的：

- 火焰图，参见下一节。
- KDAB Hotspot: https://github.com/KDAB/hotspot，[118] 一个使用与 Intel Vtune 非常相似的界面可视化 Linux perf 数据的工具。如果您使用过 Intel Vtune，KDAB Hotspot 将会非常熟悉。有些人将其用作 Intel Vtune 的替代品。
- Netflix Flamescope: https://github.com/Netflix/flamescope，[119] 显示应用程序运行时采样事件的热图。您可以观察负载行为的不同阶段和模式。Netflix 工程师使用此工具发现了一些非常微妙的性能漏洞。此外，您可以在热图上选择一个时间范围并仅为此时间范围生成火焰图。

[116] RISCV 目前还不作为官方内核的一部分支持，尽管存在供应商的自定义工具。
[117] Linux perf wiki - https://perf.wiki.kernel.org/index.php/Main_Page.
[118] KDAB Hotspot - https://github.com/KDAB/hotspot.
[119] Netflix Flamescope - https://github.com/Netflix/flamescope.

它不能做什么?

Linux perf 是一个命令行工具，缺乏 GUI，这使得过滤数据、观察工作负载行为随时间如何变化、放大运行时的一部分等变得困难。通过 `perf report` 命令提供了有限的控制台输出，这对于快速分析来说已经足够，但不如其他 GUI 分析器方便。幸运的是，正如我们刚才提到的，有一些 GUI 工具可以对 Linux `perf` 的原始输出进行后处理和可视化。

7.5 火焰图

火焰图是一种流行的性能数据可视化方式，可以直观地呈现程序中最频繁的代码路径。它允许我们看到哪些函数调用占用了最大部分的执行时间。图 52 展示了使用 Brendan Gregg 开发的开源脚本 [120] 生成的 x264: https://openbenchmarking.org/test/pts/x264 视频编码基准测试的火焰图示例。如今，几乎所有性能分析器都可以在优化分析文件会话期间收集调用堆栈的情况下自动生成火焰图。

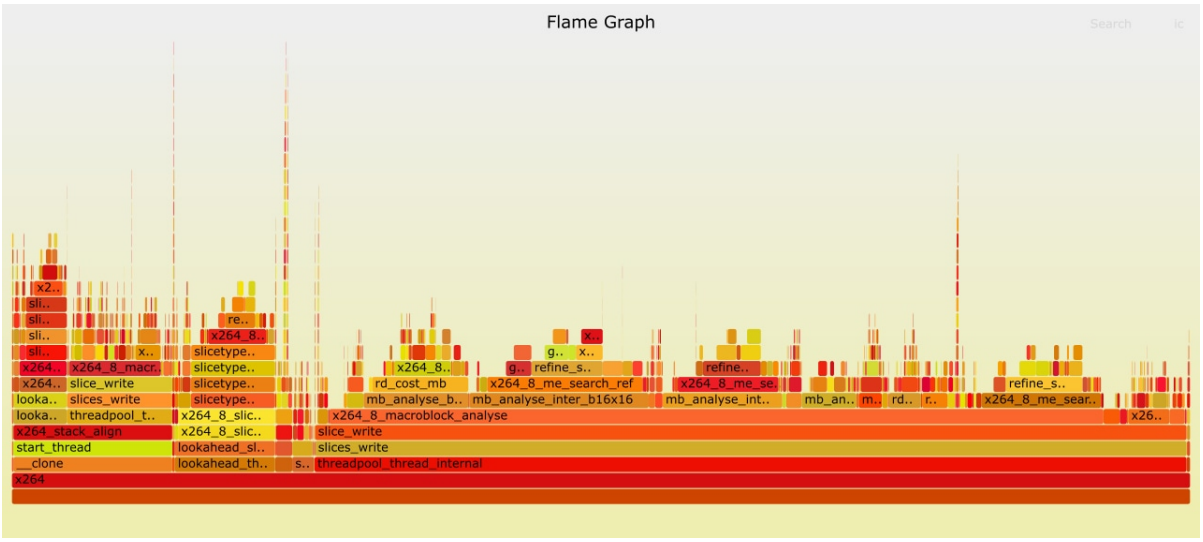


Figure 52: A Flame Graph for x264 benchmark.

在火焰图上，每个矩形（水平条）表示一个函数调用，矩形的宽度表示函数本身及其被调用者所花费的相对执行时间。函数调用是从下到上进行的，因此我们可以看到程序中最热的路径是 `x264 -> threadpool_thread_internal -> ... -> x264_8_macroblock_analyse`。函数 `threadpool_thread_internal` 及其被调用者占用了程序运行时间的 74%。但其自身时间，即函数本身花费的时间则相对较少。同样，我们可以对 `x264_8_macroblock_analyse` 进行相同的分析，它占用了 66% 的运行时间。这种可视化方式可以让您很好地直观地了解程序花费最多时间的地方。

火焰图是可交互的，您可以单击图像上的任何条形，它就会放大到特定的代码路径。您可以一直放大，直到找到与您的期望不符的地方，或者到达叶/尾函数——现在您就有了可以在分析中使用的可操作信息。另一种策略是找出程序中最热的函数（从这个火焰图中无法立即看出来），然后从下往上通过火焰图，试图理解这个最热的函数是从哪里被调用的。

7.6 Windows 事件跟踪

微软开发了一项名为 Windows 事件跟踪 (ETW) 的系统级跟踪功能。它最初旨在帮助设备驱动程序开发人员，但后来也发现它可以用于分析通用应用程序。ETW 在所有受支持的 Windows 平台（x86、x64 和 ARM）上可用，并提供相应的平台相关安装包。ETW 以结构化事件的形式记录用户和内核代码，并支持完整的调用堆栈跟踪，允许观察运行系统中的软件动态并解决许多棘手的性能问题。

[120] Brendan Gregg 的火焰图 - https://github.com/brendangregg/FlameGraph

7.6.1 如何配置它

从 Windows 10 开始，使用 `Wpr.exe` 可以录制 ETW 数据，无需任何额外下载。但是要启用系统范围的分析，您必须是管理员并启用 `SeSystemProfilePrivilege`。Windows 性能记录器工具支持一组内置录制优化分析文件，适用于常见性能问题。您可以通过创作带有 `.wprp` 扩展名的自定义性能记录器优化分析文件 XML 文件来定制您的录制需求。

如果您不仅想要录制还想查看录制的 ETW 数据，则需要安装 Windows Performance Toolkit (WPT)。您可以从 Windows SDK[121] 或 ADK[122] 下载页面下载 Windows Performance Toolkit。Windows SDK 体积庞大，您不一定需要所有部分。在我们的例子中，我们只勾选了 Windows Performance Toolkit 的复选框。您可以将 WPT 作为您自己应用程序的一部分进行再分发。

7.6.2 你能用它做什么：

- 查看 CPU 热点，可配置的 CPU 采样率从 125 微秒到 10 秒。默认为 1 毫秒，运行时开销约为 5-10%。
- 哪个线程阻塞了某个线程以及阻塞了多长时间（例如，延迟事件信号、不必要的线程睡眠等）。
- 检查磁盘处理读/写请求的速度以及谁启动了这项工作。
- 检查文件访问性能和模式（包括导致没有磁盘 I/O 的缓存读/写）。
- 跟踪 TCP/IP 堆栈如何如何在网络接口和计算机之间传输数据包。

上述所有项目都在系统范围内记录所有进程，并具有可配置的调用堆栈跟踪（内核和用户模式调用堆栈结合）。您还可以添加自己的 ETW 提供程序，将系统范围的跟踪与您的应用程序行为关联起来。您可以通过检测您的代码来扩展收集的数据量。例如，您可以在源代码中的函数中添加注入进入/离开 ETW 跟踪钩子，以测量特定方法的执行频率。

7.6.3 你不能用它做什么：

- 检查 CPU 微架构瓶颈。为此，请使用特定于供应商的工具，例如 Intel VTune、AMD uProf、Apple Instruments 等。

ETW 跟踪捕获了所有进程在系统级别的动态，这很棒，但它可能会生成大量数据。例如，捕获线程上下文切换数据以观察各种等待和延迟很容易在每分钟生成 1-2 GB 的数据。这就是为什么不实际录制大量事件数小时而不覆盖之前存储的跟踪的原因。

7.7 记录 ETW 跟踪的工具

以下是一些可以用来捕获 ETW 跟踪的工具列表：

- `wpr.exe`：命令行录制工具，它是 Windows 10 和 Windows Performance Toolkit 的一部分。
- `WPRUI.exe`：一个用于录制 ETW 数据的简单 UI，它是 Windows Performance Toolkit 的一部分。
- `xperf`：wpr 的命令行前身，是 Windows Performance Toolkit 的一部分。
- `PerfView`：[123] 一个图形化录制和分析工具，主要关注.NET 应用程序。由微软开源。
- `Performance HUD`：[124] 一个鲜为人知但功能非常强大的 GUI 工具，用于跟踪 UI 延迟、用户/句柄泄漏，以及通过实时 ETW 录制所有不平衡的资源分配，并实时显示泄漏/阻塞调用堆栈跟踪。
- `ETWController`：[125] 一个录制工具，能够录制键盘输入和屏幕截图以及 ETW 数据。还支持同时在两台机器上进行分布式分析。由 Alois Kraus 开源。

[121] Windows SDK 下载 https://developer.microsoft.com/en-us/windows/downloads/sdk-archive/

[122] Windows ADK 下载 https://learn.microsoft.com/en-us/windows-hardware/get-started/adk-install#other-adk-downloads

[123] PerfView https://github.com/microsoft/perfview

[124] Performance HUD https://www.microsoft.com/en-us/download/100813

[125] ETWController https://github.com/alois-xx/etwcontroller

- `UIForETW`：[126] 一个围绕 xperf 的包装器，具有记录 Google Chrome 问题数据的特殊选项。还可以录制键盘和鼠标输入。由 Bruce Dawson 开源。

7.7.1 查看和分析 ETW 跟踪的工具

- `Windows Performance Analyzer` (WPA)：查看 ETW 数据最强大的 UI。WPA 可以可视化和叠加磁盘、CPU、GPU、网络、内存、进程等等更多的数据源，以便全面了解您的系统行为及其执行操作。虽然 UI 非常强大，但对于初学者来说也可能相当复杂。WPA 支持插件来处理来自其他来源的数据，而不仅仅是 ETW 跟踪。可以导入 Linux/Android[127] 分析数据，这些数据是由 Linux perf、LTTNG、Perfetto 和以下日志文件格式生成的：dmesg、Cloud-Init、WaLinuxAgent、AndoidLogcat。
- `ETWAnalyzer`：[128] 读取 ETW 数据并生成聚合摘要 JSON 文件，这些文件可以在命令行进行查询、过滤和排序，或者导出为 CSV 文件。
- `PerfView`：主要用于故障排除.NET 应用程序。为垃圾回收和 JIT 编译触发的 ETW 事件被解析并作为报告或 CSV 数据轻松访问。

案例研究 - 程序启动缓慢

接下来，我们将通过一个示例，使用 ETWController 捕获 ETW 跟踪并使用 WPA 进行可视化。

问题描述: 在 Windows 资源管理器中双击下载的可执行文件时，它的启动会明显延迟。似乎有些东西延迟了进程启动。可能是什么原因？磁盘速度慢？

7.7.1.1 设置

- 下载 ETWController 以录制 ETW 数据和屏幕截图。
- 下载最新的 Windows 11 Performance Toolkit[129] 以便能够使用 WPA 查看数据。确保在系统环境对话框中将较新的 Win 11 `wpr.exe` 放置在您的路径之前，方法是将 WPT 的安装文件夹移动到 `C:\\Windows\\system32` 之前。它应该如下所示：

```
C> where wpr
C:\Program Files (x86)\Windows Kits\10\Windows Performance Toolkit\wpr.exe
C:\Windows\System32\wpr.exe
```

7.7.1.2 捕获痕迹

- 启动 ETWController。
- 选择 CSwitch 优化分析文件以跟踪线程等待时间以及其他默认录制设置。保持复选框“记录鼠标点击”和“获取循环屏幕截图”启用，以便稍后借助屏幕截图导航到慢速点。参见图 53 。
- 按“开始录制”。
- 从互联网下载一些可执行文件，解压缩它并双击可执行文件启动它。
- 之后，您可以通过按“停止录制”按钮停止分析。

第一次停止分析需要更长的时间，因为所有托管代码都会生成合成 pdb，这是一个一次性操作。分析达到已停止状态后，您可以按“在 WPA 中打开”按钮，将 ETL 文件加载到 Windows Performance Analyzer 中，并附带 ETWController 提供的优化分析文件。CSwitch 优化分析文件会生成大量数据，这些数据存储在 4 GB 的环形缓冲区中，允许您在最

[126] UIforETW https://github.com/google/UIforETW

[127] Microsoft Performance Tools Linux / Android https://github.com/microsoft/Microsoft-Performance-Tools-Linux-Android

[128] ETWAnalyzer https://github.com/Siemens-Healthineers/ETWAnalyzer

[129] Windows SDK 下载 https://developer.microsoft.com/en-us/windows/downloads/sdk-archive/

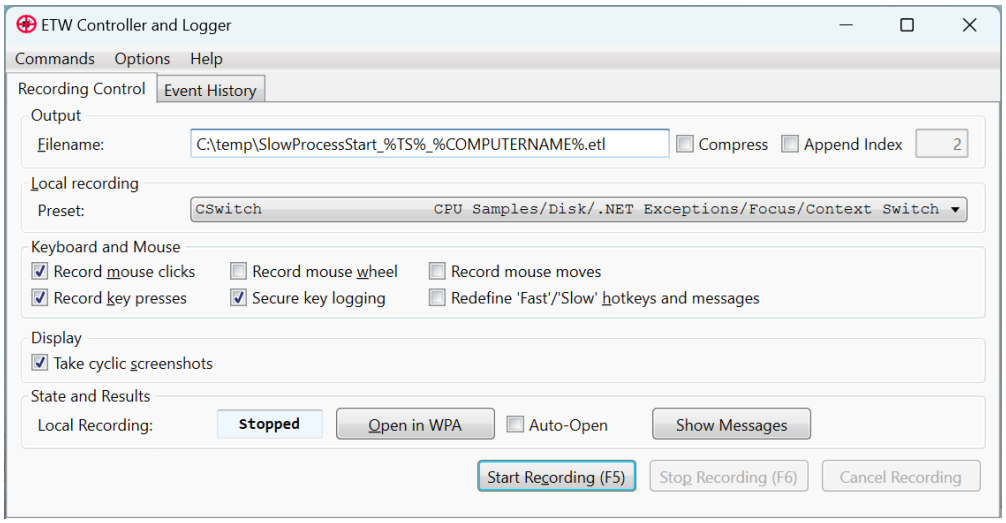


Figure 53: 使用 ETWController UI 启动 ETW 收集.

旧的事件被覆盖之前录制 1-2 分钟。有时在正确的时间点停止分析有点艺术气息。如果您遇到偶发问题，可以将录制保持启用数小时，并在事件（例如文件中由轮询脚本检查的日志条目）出现时停止录制，以在问题发生时停止录制。

Windows 支持事件日志和性能计数器触发器，允许在性能计数器达到阈值或特定事件写入事件日志时启动脚本。如果您需要更复杂的停止触发器，应该看一下 PerfView，它允许定义一个性能计数器阈值，该阈值必须达到并在那里停留 x 秒，然后停止分析。这样，随机峰值就不会再触发误报。

7.7.1.3 在 WPA 中分析

图 54 显示了在 Windows Performance Analyzer (WPA) 中打开的已录制 ETW 数据。WPA 视图分为三个部分：CPU 使用率（采样）、通用事件 和 CPU 使用率（精确）。为了理解它们之间的区别，让我们更深入地研究一下。上面 CPU 使用率（采样）图表可用于识别 CPU 时间花在何处。数据是通过定期间隔对所有正在运行的线程进行采样收集的。与其他分析工具中的热点视图非常相似。

接下来是 通用事件 视图，其中显示鼠标点击和捕获的屏幕截图等事件。请记住，我们在 ETWController 窗口中启用了拦截这些事件的功能。因为事件放在时间线上，所以很容易将 UI 交互与系统如何响应它们相关联。

底部图表 CPU 使用率（精确）使用的数据源与 采样 视图不同。虽然采样数据只会捕获正在运行的线程，但 精确 收集会考虑进程未运行的时间间隔。精确视图的数据来自 Windows 线程调度程序。它跟踪线程运行的时间和所处 CPU (CPU 使用率)、它在内核调用中被阻塞了多长时间 (等待)、它的优先级以及线程等待 CPU 可用有多长时间 (准备时间) 等。因此，精确视图不会显示顶级 CPU 耗用者。但是，这个视图对于理解某个进程被阻塞了多长时间以及 为什么 被阻塞非常有用。

现在我们熟悉了 WPA 界面，让我们观察一下图表。首先，我们可以在时间线上找到 `MouseButton` 事件 63 和 64。ETWController 将收集期间拍摄的所有屏幕截图保存在一个新建的文件夹中。分析数据本身保存在名为 `SlowProcessStart.etl` 的文件中，还有一个名为 `SlowProcessStart.etl.Screenshots` 的新文件夹。该文件夹包含屏幕截图和一个可以在浏览器中查看的 `Report.html` 文件。每个记录的键盘/鼠标交互都保存在一个以其事件编号命名的文件中，例如 `Screenshot_63.jpg`。图 55 （已裁剪）显示鼠标双击（事件 63 和 64）。鼠标指针位置标记为绿色方块，除非单击事件发生，则为红色。这使得很容易发现何时何地执行了鼠标单击。

双击标志着我们应用程序等待某事时 1.2 秒延迟的开始。在时间戳 `35.1` 时，`explorer.exe` 处于活动状态，因为它试图启动新的应用程序。但后来它没有做太多工作，应用程序也没有启动。相反，`MsMpEng.exe` 接管执行直到时间 `35.7`。到目前为止，它看起来像是在下载的可执行文件允许启动之前进行防病毒扫描。但我们不能 100% 确定 `MsMpEng.exe` 正在阻止新应用程序的启动。

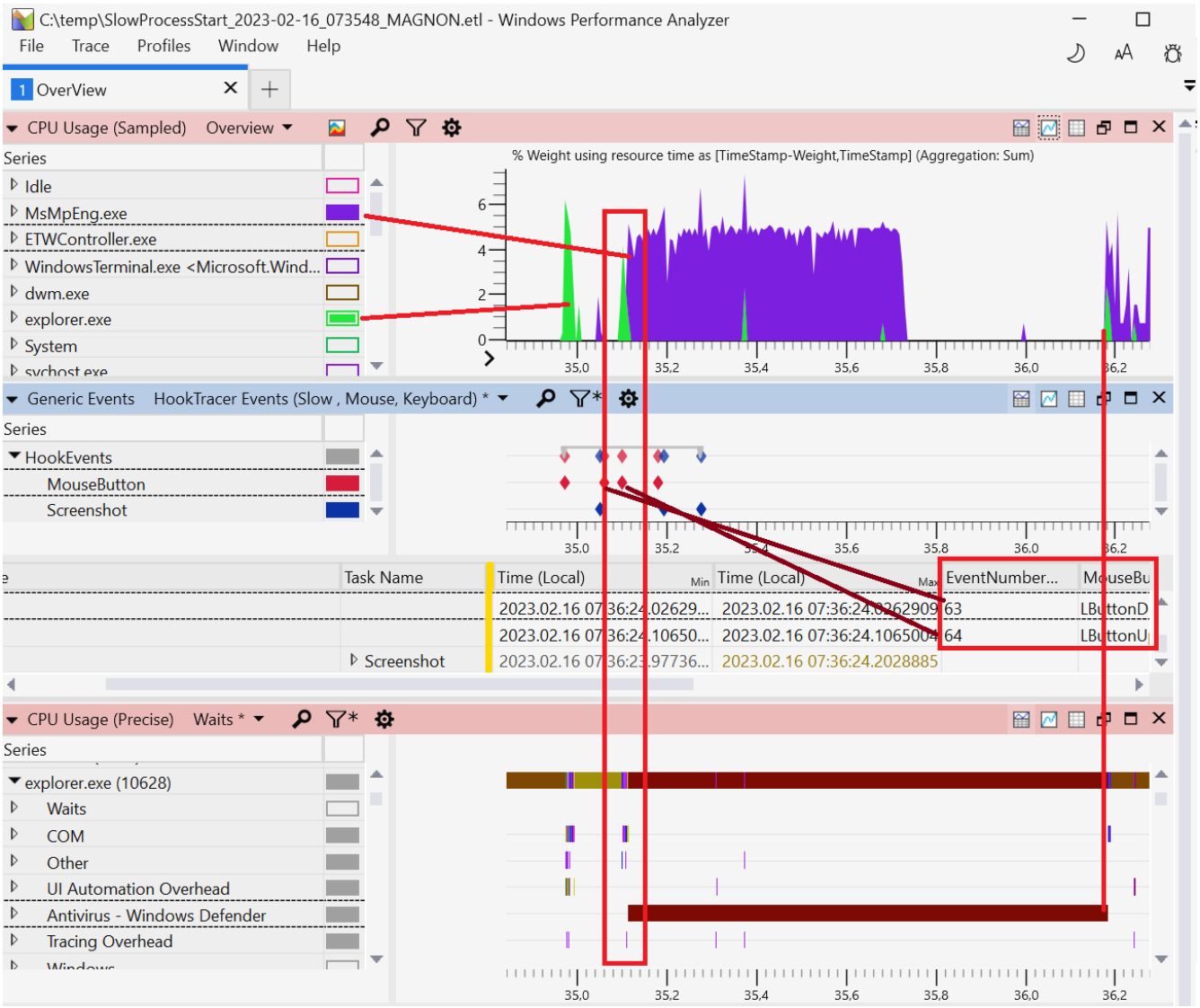


Figure 54: Windows Performance Analyzer 应用程序启动缓慢概览.

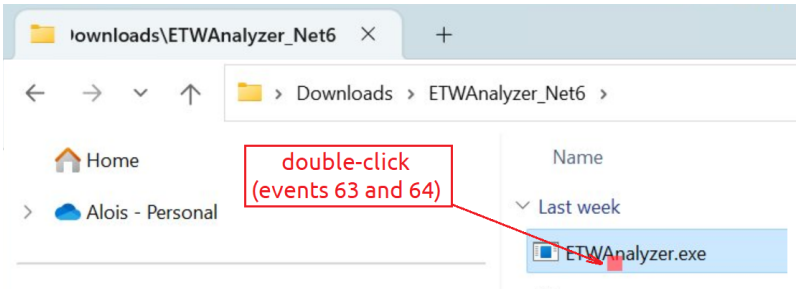


Figure 55: 使用 ETWController 捕获的鼠标点击屏幕截图.

由于我们正在处理延迟，因此我们对 CPU 使用率（精确）等待 面板上可用的等待时间感兴趣。在那里，我们找到了我们的 `explorer.exe` 正在等待的进程列表，以条形图的形式可视化，与上部面板的时间线对齐。不难发现对应于 Antivirus - Windows Defender 的长条，其等待时间为 1.068 秒。因此，我们可以得出结论，启动我们应用程序的延迟是由 Defender 扫描活动引起的。如果您深入研究调用堆栈（未显示），您将看到系统调用 `CreateProcess` 在内核中被 `WDFilter.sys`（Windows Defender 筛选器驱动程序）延迟。它会阻止进程启动，直到扫描潜在恶意文件内容为止。防病毒软件可以拦截所有内容，从而导致难以诊断的性能问题，如果没有像 ETW 这样的全面内核视图。

谜题解开了吗？嗯，还不完全是。

知道 Defender 是问题只是第一步。如果您再看上面面板，您会看到延迟并不是完全由繁忙的防病毒扫描引起的。`MsMpEng.exe` 进程从时间 `35.1` 一直活跃到 `35.7`，但应用程序并没有立即启动之后。从时间 `35.7` 到 `36.2` 有额外的 0.5 秒延迟，在此期间 CPU 大部分处于空闲状态，什么都不做。要找到根本原因，需要跟踪跨进程的线程唤醒历史，我们将在此处不介绍。最后，您会发现一个阻止性的 Web 服务调用 `MpClient.dll!MpClient::CMpSpyNetContext::UpdateSpynetMetrics`，它确实等待某个 Microsoft Defender Web 服务做出响应。如果您还启用了 TCP/IP 或套接字 ETW 跟踪，您还可以找出 Microsoft Defender 与哪个远程端点通信。因此，延迟的第二部分是由 `MsMpEng.exe` 进程等待网络引起的，这也阻止了我们的应用程序运行。

这个案例研究只展示了一个使用 WPA 可以有效分析问题的例子，但还有其他类型的问题。WPA 界面非常丰富且高度可定制。它支持自定义优化分析文件，可以按照您喜欢的方式配置图表和表格以可视化 CPU、磁盘、文件等。最初，WPA 是为设备驱动程序开发人员开发的，并且内置了一些不专注于应用程序开发的优化分析文件。ETWController 带有自己的优化分析文件 (Overview.wpaprofile)，您可以将其设置为默认优化分析文件，位于 优化分析文件 -> 保存启动优化分析文件 下，以便始终使用性能概览优化分析文件。

7.8 专业和混合性能分析器

到目前为止，我们探索的大部分工具都属于采样性能分析器。当你想识别代码中的热点时，它们非常有用，但在某些情况下，它们可能无法提供足够的粒度进行分析。根据性能分析器的采样频率和程序的行为，大多数函数可能足够快，不会出现在性能分析器中。在某些情况下，您可能想要手动定义程序哪些部分需要始终测量。例如，视频游戏渲染帧（显示在屏幕上的最终图像）平均每秒 60 帧 (FPS); 一些显示器允许高达 144 FPS。在 60 FPS 时，每个帧只有不到 16 毫秒的时间完成工作，然后才能继续下一个帧。开发人员特别关注超过此阈值的帧，因为这会导致游戏中出现明显的卡顿，从而破坏玩家体验。这种情况很难用采样性能分析器捕捉，因为它们通常只提供给定函数所花费的总时间。

开发人员创建了性能分析器，这些性能分析器提供特定环境中的一些有用功能，通常带有您可以用于手动为代码插入标记的标记 API。这允许您观察特定函数或代码块（稍后称为 zone）的性能。继续游戏行业，这个领域有一些工具：一些直接集成到游戏引擎中，如 Unreal，而另一些则作为外部库和工具提供，可以集成到您的项目中。一些最常用的性能分析器是 Tracy、RAD Telemetry、Remotery: https://github.com/Celtoys/Remotery 和 Optick: https://github.com/bombomby/optick（仅限 Windows）。接下来，我们展示了 Tracy: https://github.com/wolfpld/tracy，[130] 因为这似乎是一个更受欢迎的项目，但是这些概念也适用于其他性能分析器。

你可以用 Tracy 做什么：

- 调试程序中的性能异常，例如慢帧。
- 将慢事件与系统中的其他事件关联起来。
- 找到慢事件的共同特征。
- 检查源代码和汇编代码。
- 在代码更改后进行“前后”比较。

[130] Tracy - https://github.com/wolfpld/tracy

你不能用 Tracy 做什么：

- 检查 CPU 微架构问题，例如收集各种性能计数器。

案例研究：使用 Tracy 分析慢帧

在下面的例子中，我们使用了 ToyPathTracer: https://github.com/wolfpld/tracy/tree/master/examples/ToyPathTracer[131] 程序，这是一个简单的路径追踪器，类似于光线追踪技术，通过向场景中每个像素发射数千条射线来渲染逼真的图像。为了处理一帧，该实现将每个像素行的处理分配给一个单独的线程。

为了模拟 Tracy 可以帮助找到问题根源的典型场景，我们手动修改了代码，使某些帧比其他帧消耗更多时间。Listing 131 显示了添加了 Tracy 插桩的代码草图。请注意，我们随机选择帧来减慢速度。此外，我们还包含了 Tracy 的头文件，并向我们想要跟踪的函数添加了 `ZoneScoped` 和 `FrameMark` 宏。`FrameMark` 宏可以插入到性能分析器中标识单个帧。每个帧的持续时间将在时间线上可见，这非常有用。

代码清单：Tracy 插桩

```
#include "tracy/Tracy.hpp"

void TraceRowJob() {
  ZoneScoped;

  if (frameCount == randomlySelected)
    DoExtraWork();

  // ...
}

void RenderFrame() {
  ZoneScoped;
  for (...) {
    TraceRowJob();
  }
  FrameMark;
}
```

每个帧可以包含许多区域，由 `ZoneScoped` 宏指定。与帧类似，一个区域可以有许多实例。每次我们进入一个区域时，Tracy 都会捕获该区域新实例的统计数据。`ZoneScoped` 宏在堆栈上创建一个对象，该对象将记录对象范围内代码的运行时活动。Tracy 将此范围称为“区域”。在进入区域时，会捕获当前的时间戳。一旦函数退出，对象将记录一个新的 timestamp 并将此时间数据与函数名称一起存储。

Tracy 有两种操作模式：它可以存储所有时间数据，直到分析器连接到应用程序（默认模式），或者它只能在分析器连接时开始记录。后一个选项可以通过在编译应用程序时指定 `TRACY_ON_DEMAND` 预处理器宏来启用。如果要分发可以根据需要进行分析的应用程序，则应首选此模式。使用此选项，跟踪代码可以编译到应用程序中，并且除非附加了分析器，否则它对正在运行的程序几乎没有开销。分析器是一个单独的应用程序，它连接到正在运行的应用程序以捕获和显示实时分析数据，也称为“飞行记录器”模式。分析器可以在单独的机器上运行，这样它就不会干扰正在运行的应用程序。但是请注意，这并不意味着由插桩代码引起的运行时开销消失了——它仍然存在，但在这种情况下避免了可视化数据的开销。

[131] ToyPathTracer - https://github.com/wolfpld/tracy/tree/master/examples/ToyPathTracer

我们使用 Tracy 调试程序并找出为什么某些帧比其他帧慢的原因。数据是在一台配备 Ryzen 7 5800X 处理器的 Windows 11 机器上捕获的。该程序是用 MSVC 19.36.32532 编译的。Tracy 图形界面非常丰富，不幸的是太难容纳在一张屏幕截图中，所以我们将其分解。在顶部，有一个时间线视图，如图 56 所示，已裁剪以适合页面。它仅显示第 76 帧的一部分，渲染该帧需要 44.1 毫秒。在该图上，我们看到了在该帧期间处于活动状态的 `Main thread` 和五个 `WorkerThread`。所有线程，包括主线程，都在执行工作以推进最终图像的渲染进度。正如我们之前所说，每个线程在 `TraceRowJob` 区域内处理一行像素。每个 `TraceRowJob` 区域实例包含许多较小的区域，这些区域不可见。Tracy 会折叠内部区域并仅显示折叠实例的数量——例如，主线程中第一个 `TraceRowJob` 下方的数字 “4,109” 表示的意思。请注意，`DoExtraWork` 区域的实例嵌套在 `TraceRowJob` 区域下。这种观察已经可以导致发现，但在实际应用中可能并不那么明显。现在让我们先放下这个。

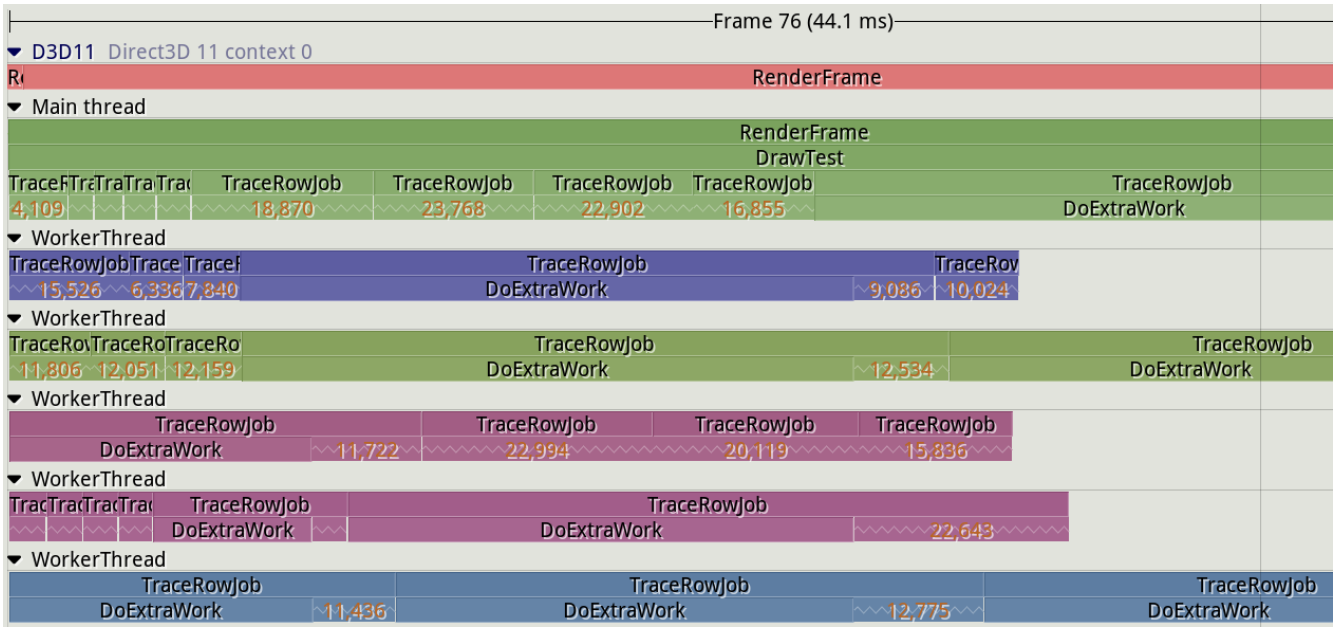


Figure 56: Tracy 主时间线视图.

在主面板的正上方，有一个直方图显示所有记录的帧的时间，请参见图 57。它可以更容易地发现可能导致卡顿的长时间运行的帧。它可以更容易地发现那些比平均时间更长才能完成的帧。在此示例中，大多数帧大约需要 33 毫秒（黄色条）。但是也有一些帧需要更长的时间，并用红色标记。如屏幕截图所示，将鼠标悬停在直方图中的条形图上时，会显示一个工具提示，显示给定帧的详细信息。在此示例中，我们显示了最后一个帧的详细信息，以绿色突出显示。

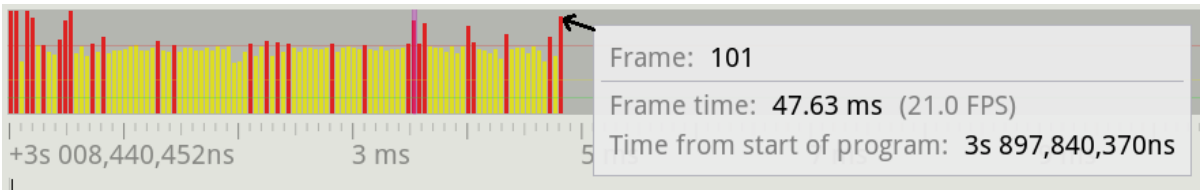


Figure 57: Tracy 帧时间.

Figure 58 展示了分析器的 CPU 数据部分。该区域显示给定线程正在哪个内核上执行，它还显示上下文切换。此部分还将显示其他正在 CPU 上运行的程序。如图像所示，将鼠标悬停在 CPU 数据视图中的给定部分上时，会显示给定线程的详细信息。详细信息包括线程运行所在的 CPU、父程序、单个线程和计时信息。我们可以看到 `TestCpu.exe` 线程在 CPU 1 上活动了 4.4 毫秒。

接下来是一个面板，提供有关程序花费时间的位置的信息，也称为热点。图 59 捕获了 Tracy 统计窗口的屏幕截图。

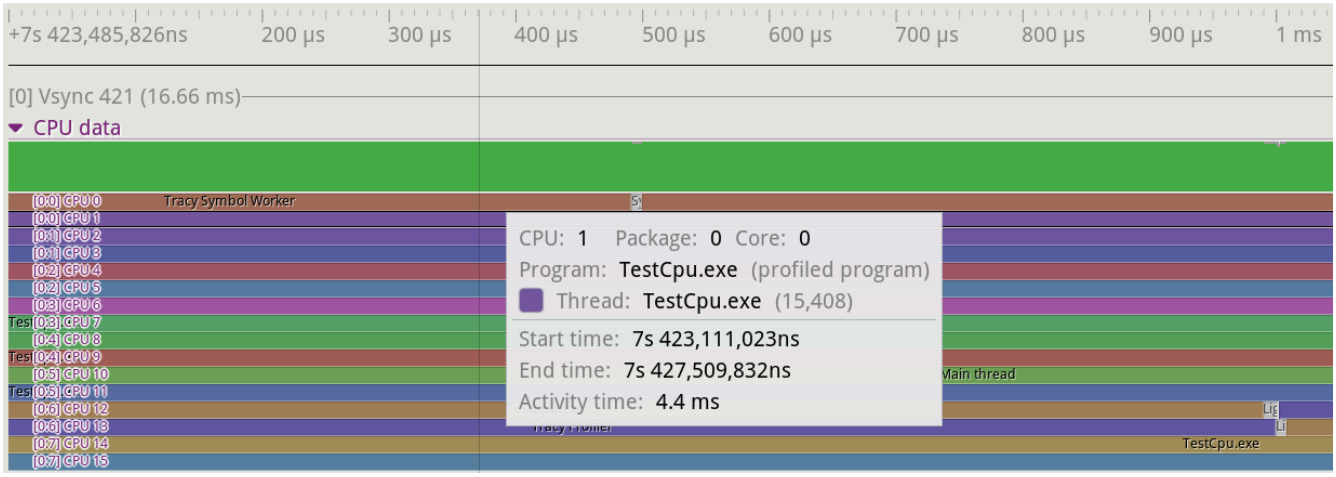
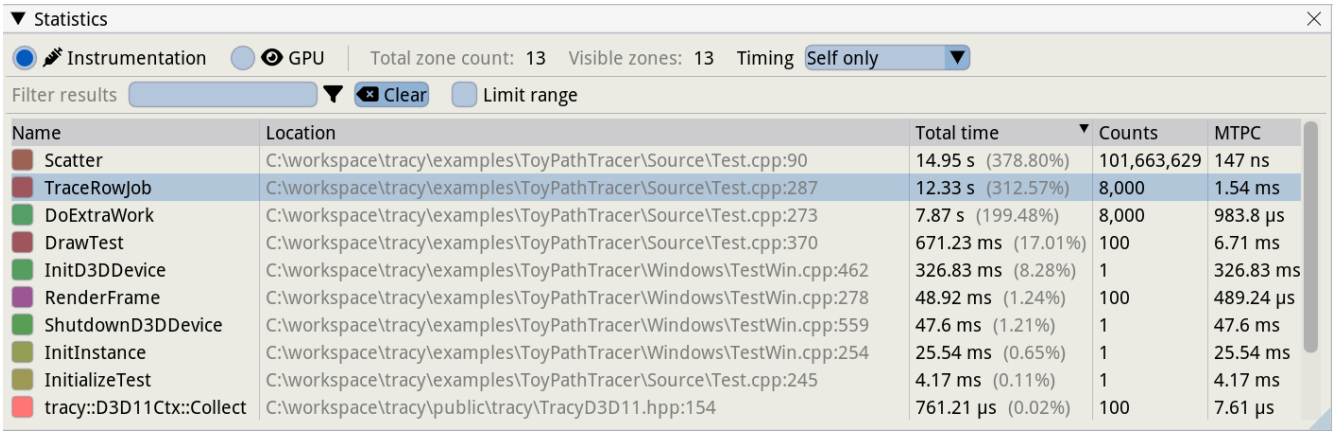


Figure 58: Tracy CPU 数据视图.

我们可以检查记录的数据，包括给定函数处于活动状态的总时间、它被调用的次数等。还可以选择主视图中的时间范围，仅过滤与该时间间隔相对应的信息。



Name	Location	Total time	Counts	MTPC
Scatter	C:\workspace\tracy\examples\ToyPathTracer\Source\Test.cpp:90	14.95 s (378.80%)	101,663,629	147 ns
TraceRowJob	C:\workspace\tracy\examples\ToyPathTracer\Source\Test.cpp:287	12.33 s (312.57%)	8,000	1.54 ms
DoExtraWork	C:\workspace\tracy\examples\ToyPathTracer\Source\Test.cpp:273	7.87 s (199.48%)	8,000	983.8 µs
DrawTest	C:\workspace\tracy\examples\ToyPathTracer\Source\Test.cpp:370	671.23 ms (17.01%)	100	6.71 ms
InitD3DDevice	C:\workspace\tracy\examples\ToyPathTracer\Windows\TestWin.cpp:462	326.83 ms (8.28%)	1	326.83 ms
RenderFrame	C:\workspace\tracy\examples\ToyPathTracer\Windows\TestWin.cpp:278	48.92 ms (1.24%)	100	489.24 µs
ShutdownD3DDevice	C:\workspace\tracy\examples\ToyPathTracer\Windows\TestWin.cpp:559	47.6 ms (1.21%)	1	47.6 ms
InitInstance	C:\workspace\tracy\examples\ToyPathTracer\Windows\TestWin.cpp:254	25.54 ms (0.65%)	1	25.54 ms
InitializeTest	C:\workspace\tracy\examples\ToyPathTracer\Source\Test.cpp:245	4.17 ms (0.11%)	1	4.17 ms
tracy::D3D11Ctx::Collect	C:\workspace\tracy\public\tracy\TracyD3D11.hpp:154	761.21 µs (0.02%)	100	7.61 µs

Figure 59: Tracy 函数统计数据.

我们展示的最后一组面板允许我们更深入地分析单个区域实例。一旦您单击任何区域实例，例如，在主时间线视图或 CPU 数据视图上，Tracy 将打开一个区域信息窗口（参见图 60，左侧面板），其中包含此区域实例的详细信息。它告诉了区域本身或其子区域消耗了多少执行时间。在此示例中，`TraceRowJob` 函数的执行耗时 19.24 毫秒，但函数本身消耗的时间不包括其被调用者，仅为 1.36 毫秒，仅占 7%。其余时间由子区域占用。

很容易发现调用 `DoExtraWork` 占据了大部分时间，16.99 毫秒中的 19.24 毫秒。请注意，这个特定的 `TraceRowJob` 实例运行时间比平均情况长 4.4 倍（图像上找到“平均时间的 437.93%”）。Bingo! 我们发现了一个慢实例，其中 `TraceRowJob` 函数由于一些额外工作而变慢。一种方法是单击 `DoExtraWork` 行以检查此区域实例。这将使用 `DoExtraWork` 实例的详细信息更新区域信息视图，以便我们可以深入了解导致性能问题的原因。此视图还显示了区域启动的源文件和代码行。因此，另一个策略是检查源代码以了解为什么当前的 `TraceRowJob` 实例比平时花费更多时间。

记得我们在图 57 上看到，还有其他慢帧。让我们看看这是否是所有慢帧的常见问题。如果我们点击“统计”按钮，它将显示“查找区域”面板（图 60，右侧）。在这里，我们可以看到聚合所有区域实例的时间直方图。这对于确定执行函数时有多少差异特别有用。查看右侧的直方图，我们看到 `TraceRowJob` 函数的中位数持续时间为 3.59 毫秒，大多数调用花费 1 到 7 毫秒之间。但是，有一些实例花费的时间超过 10 毫秒，峰值为 23 毫秒。请注意，时间轴是对数

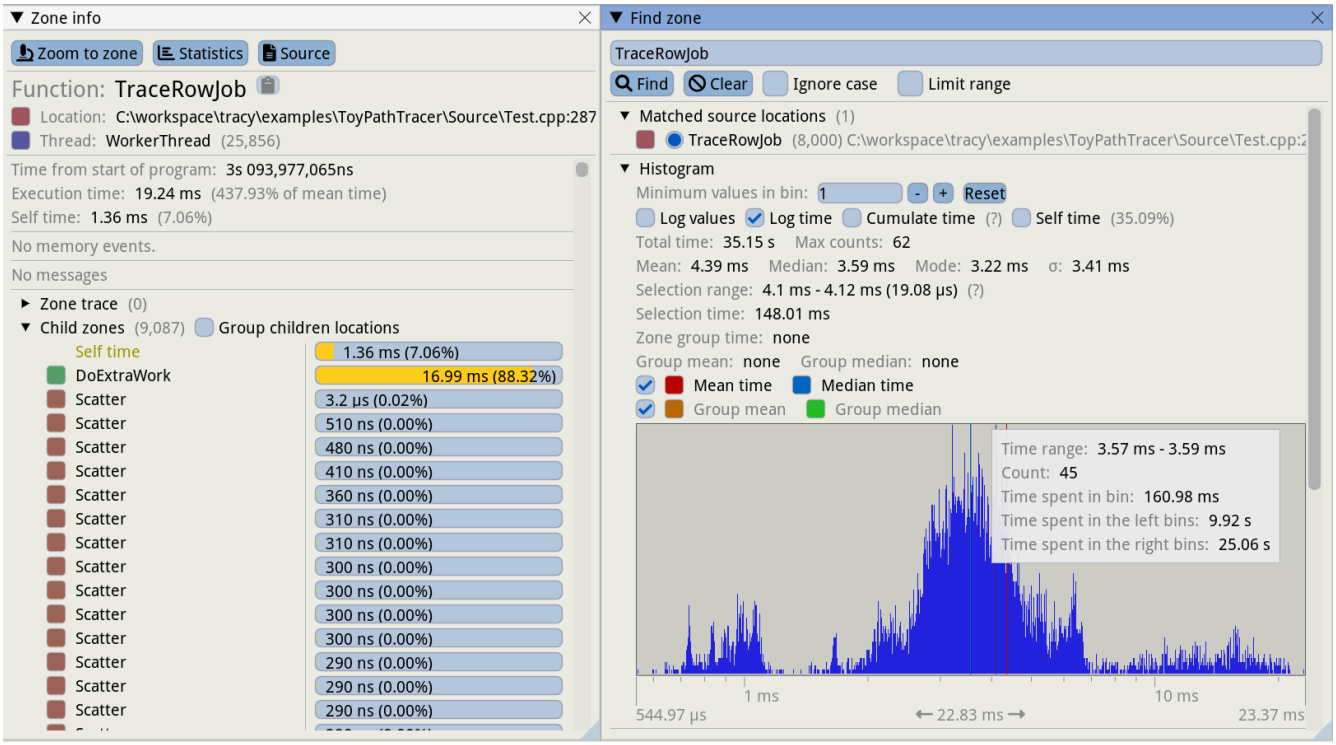


Figure 60: Tracy 区域详细信息窗口.

的。“查找区域”窗口还提供其他数据点，包括所检查区域的平均值、中位数和标准差。

现在我们可以检查其他慢实例，找到它们之间的共同点，这将帮助我们找到问题的根源。从这个视图中，您可以选择一个慢区域。这将使用该区域实例的详细信息更新区域信息窗口 (2)，然后单击“缩放到区域”按钮，主窗口将聚焦于此慢区域。从这里我们可以检查选定的 `TraceRowJob` 实例是否具有与我们刚刚分析的实例类似的特征。

7.9 Tracy 的其他功能

Tracy 不仅监控应用程序本身，还监控整个系统的性能。它还像传统采样分析器一样，报告与分析程序同时运行的应用程序的数据。该工具通过跟踪内核上下文切换（需要管理员权限）来监控线程迁移和空闲时间。区域统计数据（调用次数、时间、直方图）是准确的，因为 Tracy 会捕获每个区域的进入/退出，但系统级数据和源代码级数据是采样的。

在本节的示例中，我们使用了手动标记代码中感兴趣的区域。但这并不是开始使用 Tracy 的严格要求。您可以分析未修改的应用程序，稍后在需要时添加检测工具。Tracy 还提供许多其他功能，本文档无法全部涵盖。以下是一些值得注意的功能：

- 跟踪内存分配和锁。
- 会话比较。这对于确保更改提供了预期的益处至关重要。可以加载两个分析会话并比较更改前后区域数据。
- 源代码和汇编视图。如果可以使用调试符号，Tracy 还可以像 Intel Vtune 和其他分析器一样在源代码和相关汇编中显示热点。

与 Intel Vtune 和 AMD uProf 等其他工具相比，Tracy 无法获得相同级别的 CPU 微架构洞察力（例如，各种性能事件）。这是因为 Tracy 不利用特定于特定平台的硬件功能。

使用 Tracy 进行分析的开销取决于您激活的区域数量。Tracy 的作者提供了一些他在图像压缩程序上测量的数据点：使用两种不同的压缩方案，开销分别为 18% 和 34%。总共分析了 2 亿个区域，每个区域的平均开销为 2.25 纳秒。该

测试检测了一个非常热的功能。在其他情况下，开销会低得多。虽然可以将开销保持在较低水平，但您需要谨慎选择要检测的代码部分，尤其是在决定将其用于生产环境时。

7.10 持续性能分析

在 Chapter 5 章节中，我们介绍了进行性能分析的各种方法，包括但不限于代码注入、跟踪和采样。在这三种方法中，采样会带来相对较小的运行时开销，并且需要最少的预先工作，同时仍然可以提供宝贵的应用程序热点洞察。但这种洞察仅限于收集样本时的特定时间点 - 如果我们能为这种采样添加时间维度呢？与其只知道函数 A 在特定时间点消耗了 30% 的 CPU 周期，不如跟踪函数 A 在几天、几周甚至几个月内的 CPU 使用率变化？或者在同一时间段内检测其堆栈跟踪的变化，所有这些都在生产环境中进行？持续性能分析应运而生，将这些想法变成了现实。

什么是持续性能分析?

持续性能分析 (CP) 是一种始终处于开启状态的系统级、基于采样的性能分析器，但采样率较低，以尽量减少运行时影响。通过持续收集所有进程的数据，可以比较代码执行在不同时间段为什么会不同，甚至可以在事件发生后进行调试。CP 工具提供了宝贵的见解，可以了解哪些代码使用了最多的资源，从而使工程师能够减少生产环境中的资源使用，从而节省成本。与 Linux perf 或 Intel VTune 等典型性能分析器不同，CP 可以从应用程序堆栈一直向下定位到内核堆栈，从 任何 指定的日期和时间找到性能问题，并支持比较任意两个日期/时间的调用堆栈，以突出性能差异。

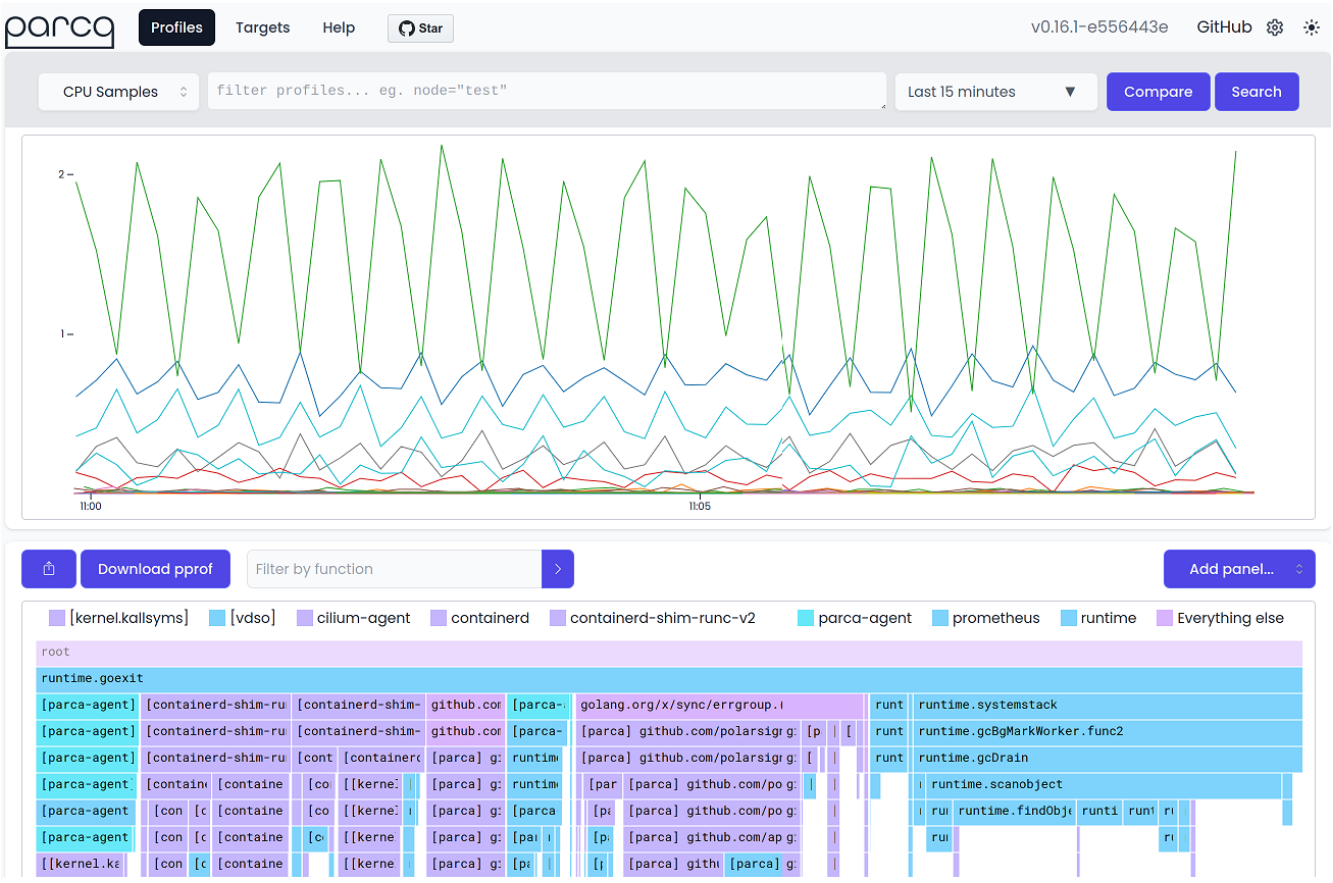


Figure 61: Parca 持续性能分析器 Web 界面截图.

为了展示典型 CP 的外观，让我们来看看开源 CP 之一 Parca: https://github.com/parca-dev/parca[132] 的 Web 界面，如图 61 所示。顶部面板显示了一个时间序列图，其中包含在从时间窗口下拉列表中选择的期间（在本例中为“最近 15 分钟”）机器上各种进程收集的 CPU 样本数量，但是为了适应页面，图片被裁剪为仅显示最后 10 分钟。默认情况下，

[132] Parca - https://github.com/parca-dev/parca

Parca 每秒收集 19 个样本。对于每个样本，它都会收集主机系统上所有进程的堆栈跟踪。分配给特定进程的样本越多，它在一段时间内的 CPU 活动就越多。在我们的例子中，您可以看到最繁忙的进程（顶部线条）表现出突发性行为，CPU 活动出现峰值和下降。如果您是此应用程序的主要开发人员，您可能会好奇为什么会发生这种情况。当您推出新版本的应用程序，突然看到分配给进程的 CPU 样本出现意外峰值，这表明出现了问题。

持续性能分析工具不仅可以更容易地发现性能变化点，还可以找到问题的根源。一旦您点击图表上的任何感兴趣点，工具就会在底部面板中显示与该周期相关的冰柱图。冰柱图是火焰图的倒置版本。使用它，您可以比较前后调用堆栈，并找到导致性能问题的确切原因。

想象一下，你将代码更改合并到了生产环境中，运行了一段时间后，收到关于间歇性响应时间峰值的报告 - 这些峰值可能与用户流量或一天中的任何特定时间相关，也可能不相关。这就是 CP 闪耀的地方。您可以拉起 CP Web UI 并搜索响应时间峰值日期和时间的堆栈跟踪，然后将它们与其他日期和时间的堆栈跟踪进行比较，以识别应用程序和/或内核堆栈级别的异常执行。这种类型的“视觉差异”直接在 UI 中支持，类似于图形“perf diff”或 差异火焰图: https://www.brendangregg.com/blog/2014-11-09/differential-flame-graphs.html。

谷歌在 2010 年的论文“Google-Wide Profiling”中介绍了这个概念，倡导了在生产环境中始终启用性能分析的价值。然而，它花了近十年才在业界获得关注：

1. 2019 年 3 月，Google Cloud 发布了其持续性能分析器。
2. 2020 年 7 月，AWS 发布了 CodeGuru Profiler。
3. 2020 年 8 月，Datadog 发布了其持续性能分析器。
4. 2020 年 12 月，New Relic 收购了 Pixie 持续性能分析器。
5. 2021 年 1 月，Pyroscope 发布了其开源的持续性能分析器。
6. 2021 年 10 月，Elastic 收购了 Optimize 及其持续性能分析器 (Prodfiler)；Polar Signals 发布了其开源的 Parca 持续性能分析器。
7. 2021 年 12 月，Splunk 发布了其 AlwaysOn Profiler。
8. 2022 年 3 月，英特尔收购了 Granulate 及其持续性能分析器 (gProfiler)。

该领域的新的参与者继续出现，既有开源版本也有商业版本。其中一些产品比其他产品需要更多的手动操作。例如，有些产品需要对源代码或优化分析文件进行更改才能开始性能分析。其他产品则针对不同的语言运行时（例如 ruby、python、golang、C/C++/Rust）需要不同的代理。最好的产品围绕 eBPF 制作了秘密武器，因此只需要安装运行时代理就可以了。

它们还支持的语言运行时数量、获取可读堆栈跟踪所需的调试符号的工作量以及除了 CPU 之外可以分析的系统资源类型（例如内存、I/O、锁定等）方面有所不同。虽然持续性能分析器在上述方面存在差异，但它们都具有为各种语言运行时提供低开销、基于采样的性能分析的共同功能，以及用于基于 Web 的搜索和查询功能的远程堆栈跟踪存储。

持续性能分析器将走向何方？Optimyze 公司的联合创始人 Thomas Dullien 开发了创新的持续性能分析器 Prodfiler，他在 QCon London 2023 上发表了主题演讲，表达了他希望拥有一款集群级工具，可以回答“为什么这个请求慢？”或“为什么这个请求昂贵？”等问题。在一个多线程应用程序中，一个特定的函数可能在性能分析中显示为 CPU 和内存消耗最高的函数，但它的职责可能完全不在应用程序关键路径上，例如，一个维护线程。与此同时，另一个函数的 CPU 执行时间非常短，在性能分析中几乎没有记录，但却可能对整体应用程序延迟和/或吞吐量产生过大的影响。典型的性能分析器无法解决这个缺点。由于持续性能分析器基本上是始终运行的性能分析器，它们也继承了同样的盲点。

值得庆幸的是，新一代的持续性能分析器已经出现，它们利用人工智能和 LLM 启发架构来处理性能分析样本，分析函数之间关系，最终高精度地找出直接影响整体吞吐量和延迟的函数和库。Raven.io 就是今天提供这种功能的一家公司。随着该领域的竞争加剧，创新能力将继续增长，使持续性能分析工具变得像典型性能分析器一样强大和稳健。

问题和练习

1. 你会使用哪些工具?
   - 场景 1：客户支持团队报告了一个客户问题：升级到新版本应用程序后，特定操作的性能下降了 10%。
   - 场景 2：客户支持团队报告了一个客户问题：一些交易完成的时间比平时长 2 倍，没有特定模式。
   - 场景 3：你正在评估三种不同的压缩算法，想知道每个算法的性能瓶颈是什么?
   - 场景 4：有一个新的闪亮库，声称比你目前项目中集成的库更快；你决定比较它们的性能。
2. 运行你每天都在使用的应用程序。根据你想要做的改进，最适合分析应用程序性能的工具是什么？练习使用这个工具。
3. 假设你在一台机器上运行多个相同的程序副本，每个副本的输入都不同。只需要分析其中一个副本，还是需要分析整个系统?

AI 回答:

1. 对于不同场景，我会推荐以下工具：
   - **场景 1**：对于新版本应用程序性能下降的问题，我会推荐使用 **Intel Vtune**（如果应用程序在基于 Intel 的系统上运行）或 **AMD uProf**（如果应用程序在基于 AMD 的系统上运行）。这些工具可以帮助识别性能下降的具体函数或代码段，以及监控 CPU 性能事件来定位问题。
   - **场景 2**：对于交易完成时间异常增长的问题，我会推荐使用 **Windows 事件跟踪 (ETW)** 或 **Linux Perf**。这些工具可以提供系统级别的性能数据，帮助分析可能的瓶颈，如线程阻塞、磁盘 I/O、网络延迟等。
   - **场景 3**：评估压缩算法性能瓶颈时，我会推荐使用 **Linux Perf** 或 **Apple Xcode Instruments**（如果是在 MacOS 上）。这些工具可以帮助分析算法在 CPU 上的执行情况，包括指令缓存命中率、分支预测错误等。
   - **场景 4**：比较不同库的性能时，我会推荐使用 **Intel Vtune** 或 **AMD uProf**，因为它们可以提供详细的性能分析，包括函数级别的 CPU 时间消耗，帮助确定哪个库在性能上更有优势。
2. 对于日常使用的应用程序，选择最适合的工具取决于应用程序的运行环境和性能改进的目标。例如，如果应用程序在 Windows 上运行，我会推荐使用 **Windows Performance Toolkit (WPT)**，它包括了 **Wpr.exe** 和 **Windows Performance Analyzer (WPA)**，这些工具可以帮助分析 CPU、内存、磁盘和网络活动。如果是在 Linux 上，**Linux Perf** 是一个很好的选择，它提供了丰富的性能监控功能。对于 MacOS，**Xcode Instruments** 是理想的选择。
   - 练习使用这些工具时，可以从简单的性能监控开始，逐步学习如何设置和解读各种性能指标，以及如何根据分析结果进行性能优化。
3. 如果在一台机器上运行多个相同的程序副本，每个副本的输入都不同，是否需要分析整个系统取决于分析的目的。如果每个副本的输入差异导致了不同的性能表现，那么分析整个系统可能会提供更全面的视角。例如，如果输入的不同导致了资源竞争或不同的执行路径，那么分析整个系统可以帮助理解这些差异如何影响性能。然而，如果目标是优化特定副本的性能，那么可能只需要分析那个特定的副本。在某些情况下，分析整个系统可以揭示系统级别的瓶颈，如 CPU 或内存限制，这些瓶颈可能影响所有副本的性能。

章节总结

- 我们快速概述了三大平台（Linux、Windows 和 MacOS）上最流行的工具。根据 CPU 供应商的不同，性能分析工具的选择也会有所不同。对于使用英特尔处理器的系统，我们推荐使用 Vtune；对于使用 AMD 处理器的系统，我们推荐使用 uProf；对于苹果平台，我们推荐使用 Xcode Instruments。
- Linux perf 可能是在 Linux 上使用最频繁的性能分析工具。它支持所有主要 CPU 供应商的处理器。但是它没有图形界面，不过有一些免费工具可以可视化 perf 的分析数据。

- 我们还讨论了 Windows 事件跟踪 (ETW)，它旨在观察运行系统中的软件动态。Linux 有一个类似的工具叫做 KUtrace: https://github.com/dicksites/KUtrace，[133] 我们这里不进行介绍。
- 此外，还有混合性能分析器，它结合了代码植入、采样和跟踪等技术。这结合了这些方法的优点，允许用户获得特定代码段非常详细的信息。本章中，我们介绍了 Tracy，它在游戏开发人员中非常流行。
- 持续性能分析已经成为监控生产环境性能的重要工具。它们可以收集系统级的性能指标，包括调用堆栈，持续时间可达数天、数周甚至数月。这些工具可以更容易地发现性能变化的点和问题的根源。

[133] KUtrace - https://github.com/dicksites/KUtrace

第二部分：源代码优化

欢迎来到本书的第二部分，我们将讨论各种低级源代码优化技术，也称为 调优。在第一部分，我们学习了如何找到代码中的性能瓶颈，这只是开发人员工作的一半。另一半是解决问题。

现代 CPU 是一个非常复杂的设备，几乎不可能预测某些代码片段的运行速度。软件和硬件性能取决于许多因素，移动部件的数量太多，超出人类思维的范围。幸运的是，通过本书第一部分讨论的所有性能监控功能，我们可以观察代码从 CPU 的角度如何运行。我们将广泛利用本书前面学到的方法和工具来指导我们的性能工程过程。

在非常高的层面上，软件优化可以分为五个类别。

- **算法优化**。理念：分析程序中使用的算法和数据结构，看看是否能找到更好的。示例：使用快速排序代替冒泡排序。
- **并行化计算**。理念：如果一个算法高度可并行化，使程序多线程化，或者考虑在 GPU 上运行。目标是同时做多件事。并发性已经在硬件和软件堆栈的所有层中使用。示例：将工作分布到多个线程上，平衡数据中心多个服务器之间的负载，使用异步 IO 以避免在等待 IO 操作时阻塞，保持多个并发网络连接以重叠请求延迟。
- **消除冗余工作**。理念：不要做你不需要的或已经完成的工作。示例：利用更多 RAM 来减少 CPU 和 IO 的使用量（缓存、记忆化、查找表、压缩），预计算已知编译时值，将循环不变计算移出循环，传引用 C++ 对象以避免传值引起的过度复制。
- **批量处理**。理念：聚合多个类似的操作并一次性执行，从而减少重复操作的开销。示例：发送较大的 TCP 数据包而不是许多小的数据包，分配大块内存而不是为数百个微小对象分配空间。
- **排序**。理念：重新排序算法中的操作序列。示例：更改数据布局以启用顺序内存访问，根据 C++ 多态对象的类型对数组进行排序以更好地预测虚函数调用，将热门函数分组并将其放置在二进制文件中更近的位置。

本书讨论的许多优化属于多个类别。例如，我们可以说向量化是并行化和批量处理的结合；循环阻塞（tiling）是批量处理和消除冗余工作的体现。

为了使图片完整，让我们也列出其他一些也许明显但仍然相当合理的加速方法：

- **使用另一种语言重写代码**：如果程序是用解释性语言（python、javascript 等）编写的，将其性能关键部分重写成开销更少的语言，例如 C++、Rust、Go 等。
- **调整编译器选项**：检查您是否至少使用了以下三个编译器标志：`-O3`（启用与机器无关的优化）、`-march`（启用针对特定 CPU 架构的优化）、`-flto`（启用过程间优化）。但不要就此止步，还有许多其他选项会影响性能。我们将在以后的章节中研究一些。可以考虑挖掘最佳选项集应用程序，可用的商业产品可以自动完成此过程。
- **优化第三方软件包**：绝大多数软件项目利用专有和开源代码层。这包括操作系统、库和框架。您还可以通过替换、修改或重新配置其中之一来寻求改进。
- **购买更快的硬件**：显然，这是一个与成本相关的业务决策，但有时它是其他选项都已用尽时唯一提高性能的方法。当您识别出应用程序中的性能瓶颈并清楚地传达给上层管理人员时，购买硬件更容易获得批准。例如，一旦您发现内存带宽限制了您的多线程程序的性能，您可能会建议购买具有更多内存通道和 DIMM 插槽的服务器主板和处理器。

算法优化

标准算法和数据结构并不总是适用于性能关键型工作负载。例如，链接列表基本上已被淘汰，取而代之的是“扁平”数据结构。传统上，链接列表的每个新节点都是动态分配的。除了可能调用许多昂贵的内存分配之外，这可能导致列表的所有元素都分散在内存中。遍历这样的数据结构不利于缓存。尽管算法复杂度仍然是 O(N)，但实际上，其运行时间会比普通数组差得多。一些数据结构，比如二叉树，有自然链表式的表示，所以用指针追逐的方式实现它们可能很诱人。然而，存在更高效的“扁平”版本，参见 `boost::flat_map`、`boost::flat_set`。

在选择算法时，您可能会快速选择最流行的选项然后继续… 即使它可能不是您特定情况下的最佳选择。例如，您需要在已排序数组中找到一个元素。大多数开发人员考虑的第一个选项是二分搜索，对吧？它非常知名，并且在算法复杂度方面是最佳的，O(logN)。如果我告诉你数组保存 32 位整数，并且数组的大小通常很小（小于 20 个元素），你会改变你的决定吗？最终，测量应该指导您的决策，但二分搜索会受到分支预测错误的影响，因为每个元素值的测试都有 50% 的可能性为真。这就是为什么在小型数组上，即使线性扫描具有更差的算法复杂度，它通常也更快的原因。

数据驱动优化

性能调优最重要的技术之一称为“数据驱动”优化，它基于程序处理的数据进行内省。该方法侧重于数据的布局及其在整个程序中的转换。这种方法的一个经典例子是数组结构到结构数组的转换，如 Listing 132 所示。

代码清单：SOA 到 AOS 转换。

```
struct S {
  int a;
  int b;
  int c;
  // other fields
};
S s[N];    // AOS

<=>

struct S { // SOA
  int a[N];
  int b[N];
  int c[N];
  // other fields
};
```

数据驱动开发 (DDD) 的主要思想是研究程序如何访问数据（数据在内存中的布局，访问模式），然后相应地修改程序（改变数据布局，改变访问模式）。

> **Personal Experience:** 实际上，我们可以说所有的优化在某种程度上都是数据驱动的。即使是我们将在下一节看到的转换，也是基于我们从程序执行中获得的一些反馈：函数调用次数，分支是否被采取，性能计数器等。

DDD 的另一个广泛示例是“小尺寸优化”。它的想法是静态预分配一定量的内存来避免动态内存分配。对于元素上限可以很好预测的中小型容器来说，它尤其有用。现代 C++ STL 的 `std::string` 实现将前 15-20 个字符保存在堆栈上分配的缓冲区中，只有更长的字符串才会在堆上分配内存。LLVM 的 `SmallVector` 和 Boost 的 `static_vector` 也是这种方法的其他例子。

低级优化

性能工程是一种艺术。就像任何艺术一样，可能的情况是无穷无尽的。不可能涵盖所有可以想象的优化。接下来的几个章节主要讨论针对现代 CPU 架构的优化。

在我们跳入特定的源代码优化技术之前，需要做一些注意事项。首先，避免优化糟糕的代码。如果一段代码存在高

级性能低下问题，你不应该对其应用机器特定的优化。始终先关注修复主要问题。只有当你确定算法和数据结构针对你要解决的问题是最佳的，然后尝试应用低级改进。

其次，请记住，您实施的优化可能并非在所有平台上都受益。例如，循环阻塞取决于系统内存层次结构的特征，尤其是 L2 和 L3 缓存的大小。因此，针对具有特定 L2 和 L3 缓存大小的 CPU 调整的算法可能无法很好地适用于缓存更小的 CPU。在您的应用程序将运行的平台上测试更改很重要。

接下来的四章按照 TMA 分类进行组织（参见 Section 6.1）:

- 第 8 章. 优化内存访问 - `TMA:MemoryBound` 类别
- 第 9 章. 优化计算 - `TMA:CoreBound` 类别
- 第 10 章. 优化分支预测 - `TMA:BadSpeculation` 类别
- 第 11 章. 机器代码布局优化 - `TMA:FrontEndBound` 类别

此分类背后的思想是为使用 TMA 方法进行性能工程工作的开发人员提供一个清单。每当 TMA 将性能瓶颈归因于上述类别之一时，您可以随时参考相应的章节以了解您的选项。

第 14 章涵盖了不属于上述任何类别的其他优化领域。第 15 章解决了一些优化多线程应用程序中常见的难题。